Authors

The work can be done in groups of up to 3 students. Please complete the following fields with your group number and list your names along with ISU ID numbers.

Technical Vision

1. Сухов Николай Михайлович, 368876
2. Иванов Никита Михайлович, 368225
3. Смирнов Алексей Владимирович, 409578

The task and guidelines were prepared by Andrei Zhdanov and Sergei Shavetov, ITMO University, 2025.

Practical Assignment No 2. Images Geometric Transformations

Studying of mapping main types and using geometric transformations for spatial image correction.

Task 1. Simplest geometric transformations

Select an arbitrary image. Perform linear and nonlinear transformations over it (conformal, affine and projective mappings).

Images geometric transformations imply a spatial change in the location of pixels set with integer coordinates (x, y) to another set with coordinates (x', y'), and the pixels intensity doesn't changes. In two-dimensional plane geometric transformations, as a rule, Euclidean space is used P^2 with an orthonormal Cartesian coordinate system. In this case, the image pixel corresponds to a pair of Cartesian coordinates, which are interpreted as a two-dimensional vector represented by a segment from a point $(0, 0)$ to a point $X_i = (x_i, y_i)$. Two-dimensional transformations on a plane can be represented as the points movement corresponding to a pixels plurality.

For generality with further transformations, we will use *homogeneous coordinates*. If homogeneous coordinates of a point are multiplied by a non-zero scalar then the resulting coordinates represent the same point. Due to this the required coordinates number to represent points is always one more than the space dimension P^n, in which these coordinates are used. For example, to represent a point $X = (x, y)$ on a plane in two-dimensional space P^2 three coordinates are required $X = (x, y, w)$. Let's illustrate this with the following example:

$$\begin{bmatrix} x' \\ y' \\ w \end{bmatrix} = w \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Leftrightarrow \begin{bmatrix} x' & y' & w \end{bmatrix} = \begin{bmatrix} x & y & 1 \end{bmatrix} w$$

where w is an arbitrary scalar factor, $x=\frac{x'}{w}$, $y=\frac{y'}{w}$.

Any linear transformation of the plane can be described using triples of homogeneous coordinates and third order matrices. Thus, geometric transformations are matrix transformations, and the sets pixel coordinates of the transformed and original images are related by the following matrix relation or in a row form $X'=X\,T$, or in a column form $X'=T^T X$. Let's rewrite these relations:

$$\begin{bmatrix} x' & y' & w' \end{pmatrix} = \begin{bmatrix} x & y & w \end{pmatrix} \cdot \begin{bmatrix} A & D & G \\ B & E & H \\ C & F & I \end{pmatrix}$$

$$\Leftrightarrow \begin{bmatrix} x' \\ y' \\ w' \end{pmatrix} = \begin{bmatrix} A & B & C \\ D & E & F \\ G & H & I \end{pmatrix} \cdot \begin{bmatrix} x \\ y \\ w \end{pmatrix}$$

Point with Cartesian coordinates (x, y) in homogeneous coordinates is written as $(x, y, 1)$:

$$\begin{bmatrix} x' & y' & 1 \end{pmatrix} = \begin{bmatrix} x & y & 1 \end{pmatrix} \cdot \begin{bmatrix} A & D & 0 \\ B & E & 0 \\ C & F & 1 \end{pmatrix}$$

$$\Leftrightarrow \begin{bmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{bmatrix} A & B & C \\ D & E & F \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{bmatrix} x \\ y \\ 1 \end{pmatrix}$$

So, the transformation formula can be rewritten as following system equations:

$$\begin{cases} x' = A\,x + B\,y + C, \\ y' = D\,x + E\,y + F \end{cases}$$

OpenCV affine transformation matrices

OpenCV used $2\,D$ vectores as a pixel coordinates and stores transformation matrix in a form of 2×2 transformation matrix and a shift vector. Also, vectors are stored columns, so the above tranformation folrmula should be rewritten as follows:

$$\begin{bmatrix} x' \\ y' \end{pmatrix} = T \cdot \begin{bmatrix} x \\ y \end{pmatrix} + V$$

Moreover, the matrix T and vector V are comined in a single 2×3 matrix to form a single OpenCV transformation matrix:

$$T' = \begin{bmatrix} T & V \end{pmatrix}$$

To match this definition with the one given above, we have to set our matrix and vector as following:

$$T=\begin{bmatrix} A & B \\ D & E \end{bmatrix}, V=\begin{bmatrix} C \\ F \end{bmatrix}, T'=\begin{bmatrix} A & B & C \\ D & E & F \end{bmatrix}$$

Then after substitution we get:

$$\begin{bmatrix} x' \\ y' \end{bmatrix}=\begin{bmatrix} A & B \\ D & E \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}+\begin{bmatrix} C \\ F \end{bmatrix}$$

1.1 Preparation

First, we need to add some imports for OpenCV to work correctly.

```
# Import OpenCV library both as cv and cv2
import cv2
import cv2 as cv
# Import math for trigonometric functions
import math
# Import NumPy library both as np and numpy
import numpy
import numpy as np
# Import sys for general constants
import sys
```

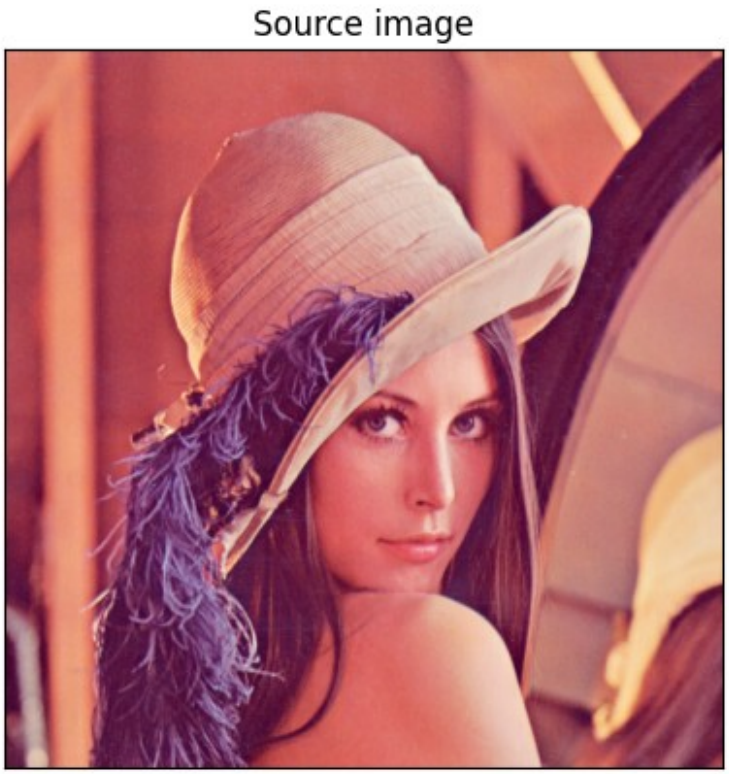
Also we will import `ShowImages()` function from the first practical assignment to use it here. It is placed in `pa_utils`.

```
from pa_utils import ShowImages
```

1.2 Read and display an image

Now let's open our image which we will use during the current task. We will open it in BGR and convert to grayscale. We will use previously introduced functions for image display.

```
# Read an image from file in BGR
fn = "images/lena_color.png"
I = cv.imread(fn, cv.IMREAD_COLOR)
if not isinstance(I, np.ndarray) or I.data == None:
  print("Error reading file \"{}\"".format(fn))
else:
  # Display it
  ShowImages([("Source image", I)])
```



1.3 Linear transformations

Linear transformation (also called **Linear mapping**) --- is a mapping that preserves the infinitesimal figures shape and the angles between curves at their intersection points. The main linear mapping are Euclidean transformations. These transformations include shift, flip, uniform scaling, and rotation. Linear transformations are a affine transformations subset.

1.3.1 Image shift

The simplest image transformation is an image shift. System equations and coordinates transformation matrix T in case of image *shift* $A=E=1$, $B=D=0$ take the form:

$$\begin{pmatrix} x'=x+C \\ y'=y+F \end{pmatrix}$$

Which can be also written in the vector form

$$\begin{bmatrix} x' & y' & 1 \end{pmatrix} = \begin{bmatrix} x & y & 1 \end{pmatrix} T$$

And gives us a very simple transformation matrix

$$T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ C & F & 1 \end{pmatrix}$$

where C and F are the shift values along axes $O\,x$ and $O\,y$ correspondingly.

As we use OpenCV, we should define this matrix in the following form:

$$T = \begin{bmatrix} 1 & 0 & C \\ 0 & 1 & F \end{bmatrix},$$

In OpenCV implementation for the Python programming language the transformation matrix is a simple `numpy` array with dimensions 3×2, so it can be created by calling the `numpy.float32()` function. Then, the `cv2.warpAffine()` function should be used to perform the affine transformation. Its first argument is an image to transform, the second one is the transformation matrix and the third one is a resolution to be used when creating the resulting image. It returns a transformed image as a result. As the image resolution may be changed during the image transformation, this could be accounted and given as a third image parameter. In case of image shift, the resolution is changed by the shift value, so we will account this by increasing the image size.

Please note Since numpy arrays are used to store images, so all methods that are applicable to numpy arrays can be executed for images as well. However this results in a little problem as images and matrices has a different order of width (number of columns) and height (number of rows) definition way. In general, image is defined by $width \times height$, however matrix is defined by $rows \times columns$, so in the image shape parameter `I.shape` the image resolution is stored as numbers of rows and columns, but OpenCV functions, e.g, the transformation function, the resolution is defined as width by height, so it should get the numbers of columns and number of rows.

First, let's try shifting an image without setting the desired transformed image resolution.

```
# Create transformation matrix and call cv.warpAffine
shift_x, shift_y = (50, 100)
T = np.float32([[1, 0, shift_x],
                [0, 1, shift_y]])
Ishift = cv.warpAffine(I, T, I.shape[0:2])

# Display it
ShowImages([("Source image", I),
            ("Image shift", Ishift)])
```

Source image

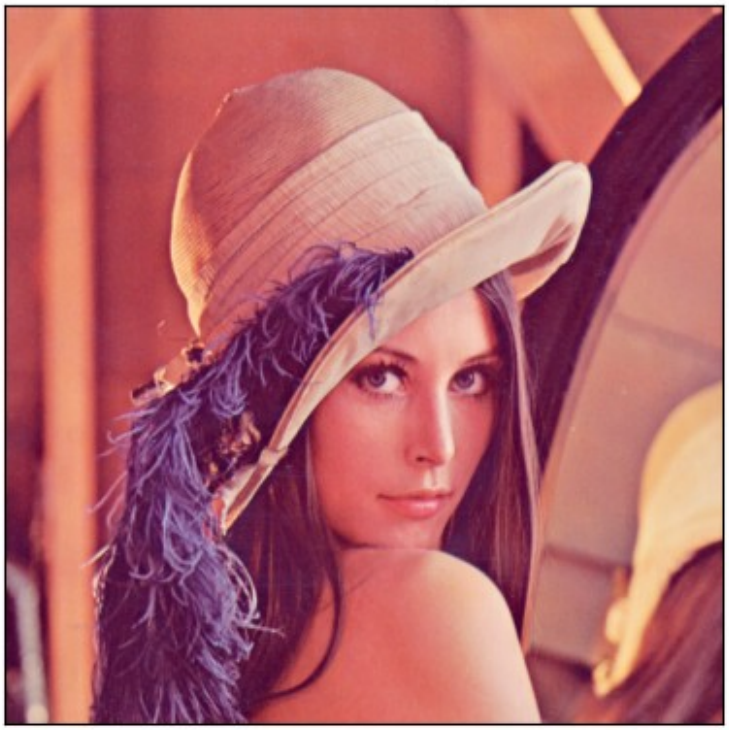
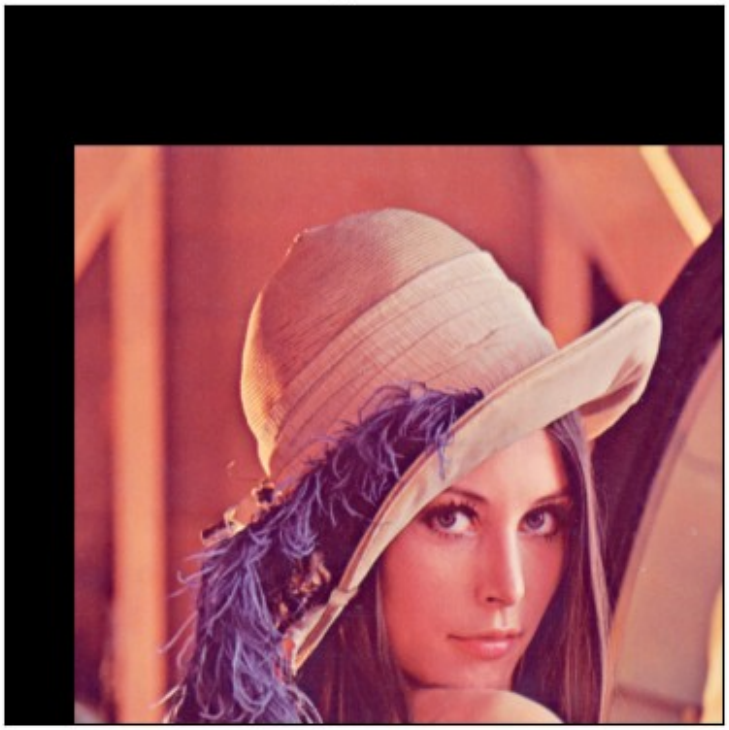


Image shift

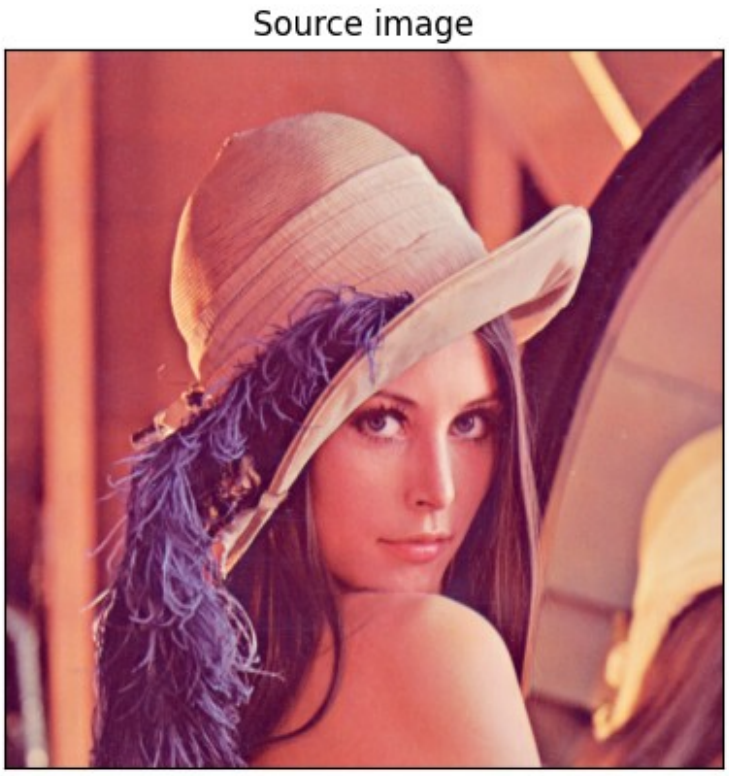


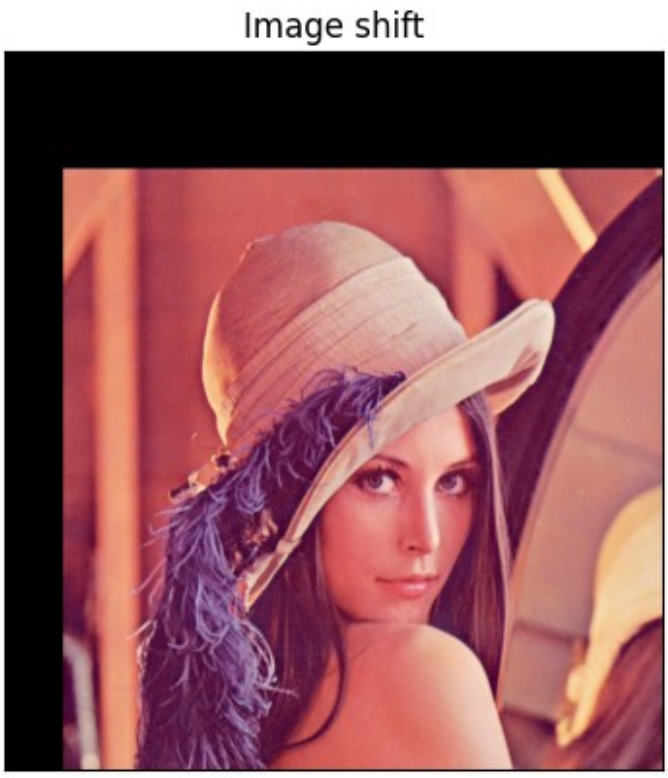
As you can see, some part of the image was lost due to to fact that transformed image data now moved outside of the image canvas. To correct it we should know the desired transformed

image resolution and pass the correct value as a third argument to the `cv2.warpAffine()` function. The transformed image resolution is a tuple for the transformed image width and height. In current case it should be equal to $(width+50, height+100)$.

Let's account for it as well.

```
# Create transformation matrix and call cv.warpAffine
shift_x, shift_y = (50, 100)
T = np.float32([[1, 0, shift_x],
                [0, 1, shift_y]])
Ishift2 = cv.warpAffine(I, T, (I.shape[1] + shift_x, I.shape[0] +
shift_y))

# Display it
ShowImages([("Source image", I),
            ("Image shift", Ishift2)])
```





As you see now image is shifted and due to a resolution change no image information is lost.

1.3.1.1 Self-work

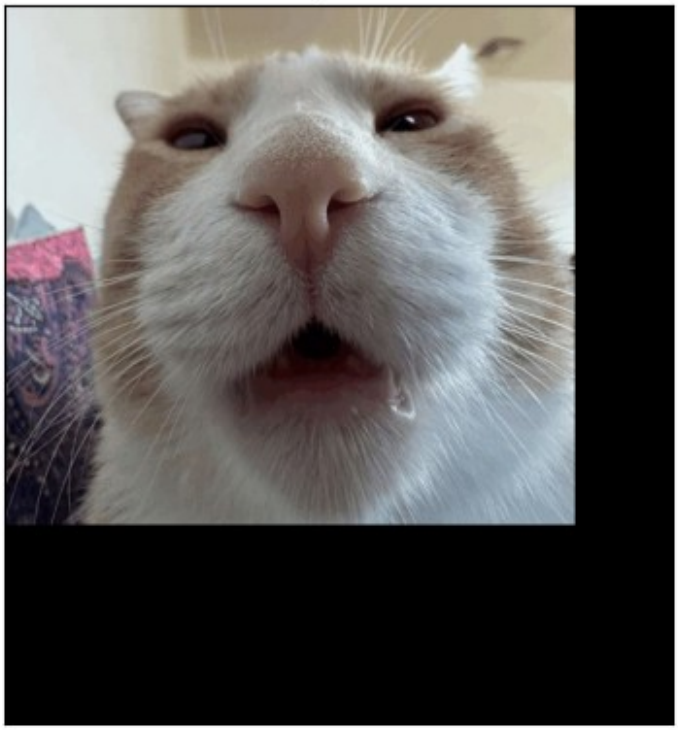
Self-work

Shift an image to the left and up by $(-40, -80)$. Don't forget to change the image resolution when transforming an image. It should be decreased by the shift vector.

```
I_cat = cv.imread("images/cat.jpg", cv.IMREAD_COLOR)

shift_x3, shift_y3 = (-40, -80)
T3 = np.float32([[1, 0, shift_x3],
                 [0, 1, shift_y3]])

Ishift3 = cv.warpAffine(I_cat, T3, (I_cat.shape[1] + np.abs(shift_x3),
I_cat.shape[0] + np.abs(shift_y3)))

ShowImages([("Source image", I_cat),
            ("Image shift", Ishift3)])
```

Source image

Image shift



1.3.2 Image flip

The transformation equation and coordinates transformation matrix T parameters in case of *image flip* around the axis $O\,x$ (along axis $o\,Y$) are $A=1$, $E=-1$, $B=C=D=F=0$. Then equations and matrix will take the form:

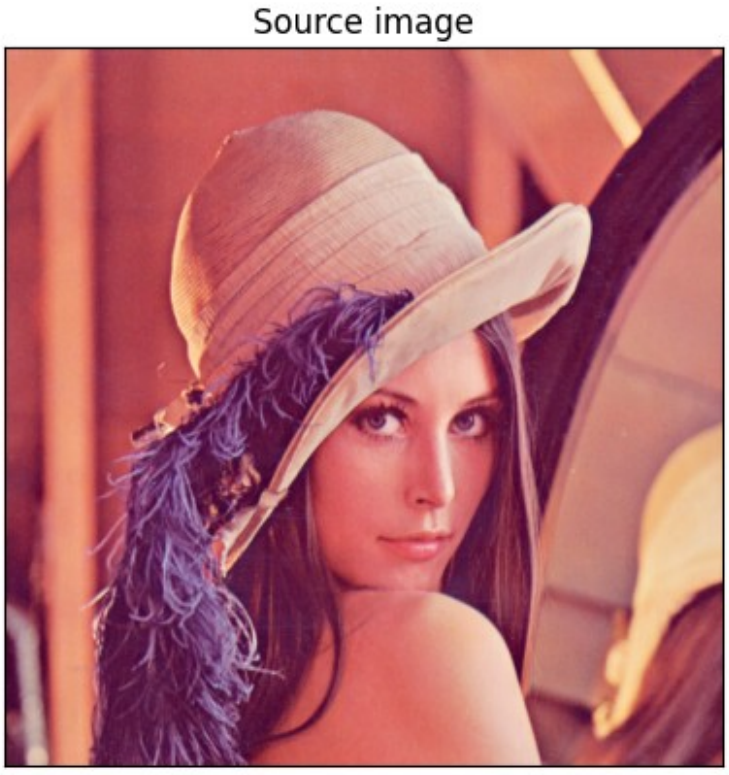
$$\left.\begin{cases} x'=x \\ y'=-y \end{cases}\right\} \Rightarrow T=\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

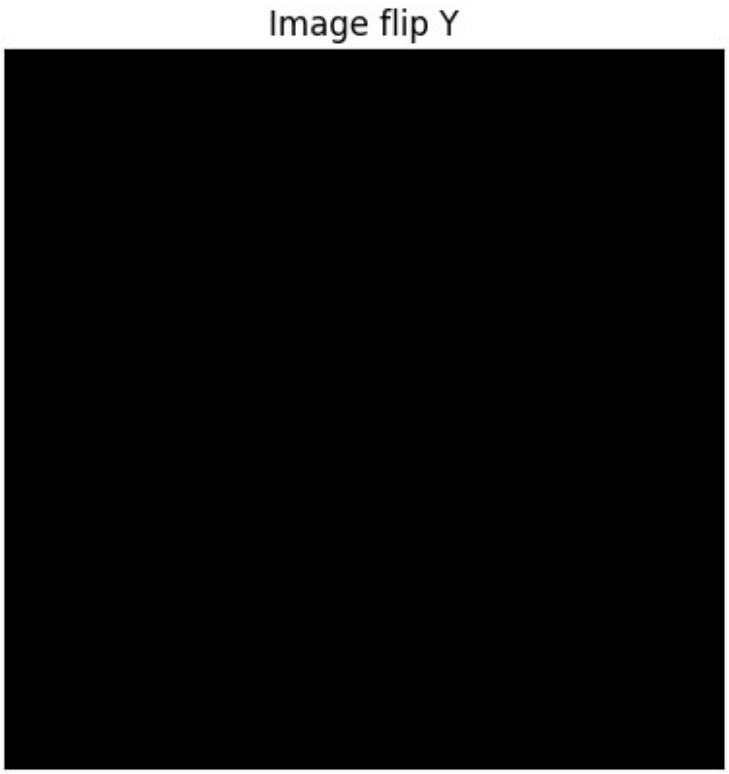
In case of OpenCV the transformation matrix should be defined as:

$$T=\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$

Let's run it.

```
# Flip around Ox
T = np.float32([[1, 0, 0],
                [0, -1, 0]])
Iflip_y = cv.warpAffine(I, T, I.shape[:2][::-1])

# Display it
ShowImages([("Source image", I),
            ("Image flip Y", Iflip_y)])
```

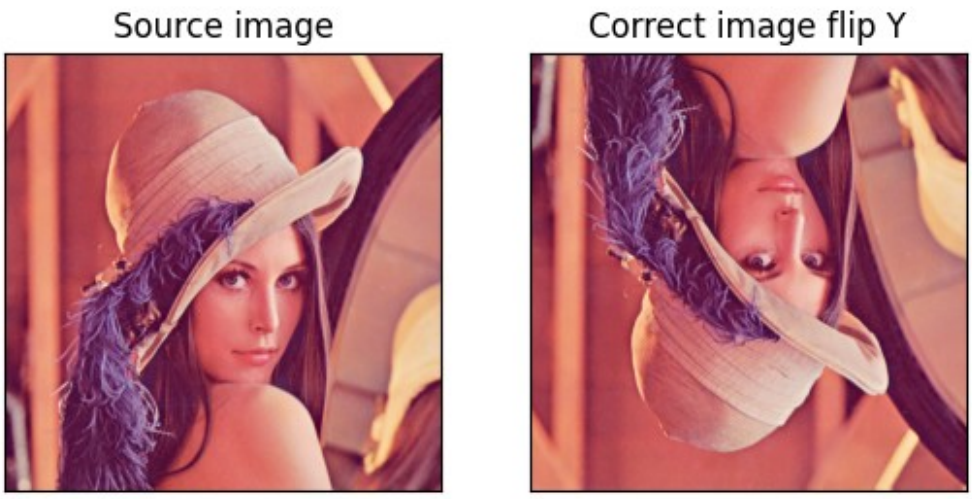




As you see, we got a black image. This happened because after flipping an image, all pixel coordinates became negative and got outside of the image area. So, we have to shift them to the image area by shifting our image right by $rows - 1$:

$$T = \begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & I.rows - 1 \end{bmatrix}$$

```
# Flip around Ox
T = np.float32([[1, 0, 0],
                [0, -1, I.shape[0] - 1]])
Iflip_y_2 = cv.warpAffine(I, T, I.shape[:2][::-1])

# Display it
ShowImages([("Source image", I),
            ("Correct image flip Y", Iflip_y_2)], 2)
```



OpenCV also provides a function for fast flipping an image called `flip()`. It takes one parameter in addition to an image to flip. The value 0 means flipping along $O\,x$ axis, 1 is for flipping along $O\,y$ axis, and -1 is to flip along both axes.

Let's try doing the same flip with OpenCV built-in function.

```
# Flip around Ox
Iflip_y_ocv = cv.flip(I, 0)

# Display it
ShowImages([("Source image", I),
            ("Built-in image flip Y", Iflip_y_ocv)], 2)
```

1.3.2.1 Self-work

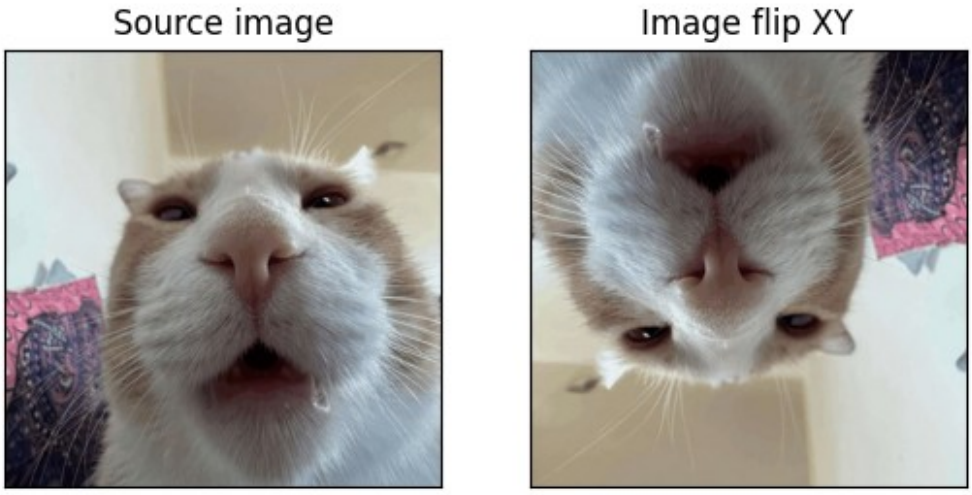
Self-work

Flip an image along both axis ($O\,x$ and $O\,y$). Implement two solutions:

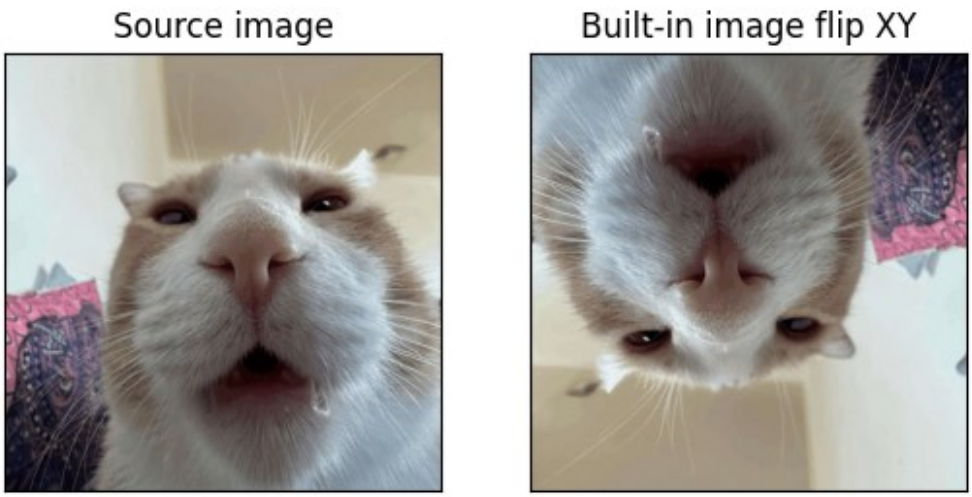
1. Create the transformation matrix and apply it with `cv2.warpAffine()` function (you may either create a single transformation matrix or apply transformation two times: for $O\,x$ and $O\,y$ axis);
2. Use the OpenCV built-in `cv2.flip()` function.

Compare the results, they should match.

```
Iflip_xy = np.zeros_like(I)

T_flip = np.float32([[-1, 0, I_cat.shape[1] - 1],
                     [0, -1, I_cat.shape[0] - 1]])
Iflip_xy = cv.warpAffine(I_cat, T_flip, I_cat.shape[:2][::-1])

ShowImages([("Source image", I_cat),
            ("Image flip XY", Iflip_xy)], 2)
```




```
Iflip_y_ocv = cv.flip(I_cat, -1)

ShowImages([("Source image", I_cat),
            ("Built-in image flip XY", Iflip_y_ocv)], 2)
```



Did your rotation results match? Don't forget to check it. There might be a little difference due to a different floating point values processing.

1.3.3 Uniform image scale

The transformation equation and coordinates transformation matrix T parameters in case of *image scale* by α along $o\,X$ axis and β along $o\,Y$ axis are $A=\alpha$, $E=\beta$, $B=C=D=F=0$. Then equations and matrix will take the form:

$$\left.\begin{cases} x'=\alpha\,x, \alpha>0 \\ y'=\beta\,y, \beta>0 \end{cases}\right\} \Rightarrow T=\begin{bmatrix} \alpha & 0 & 0 \\ 0 & \beta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

If $\alpha<1$ and $\beta<1$, then the image decreases in size, if $\alpha>1$ and $\beta>1$ then it increases. If $\alpha\neq\beta$, then the proportions will not be the same in width and height. In general, this mapping will be affine, not linear.

In case of using OpenCV libraries the transformation matrix should be defined as:

$$T=\begin{bmatrix} \alpha & 0 & 0 \\ 0 & \beta & 0 \end{bmatrix},$$

When resizing an image we should also change the image size as well, so the new image size will be $(width\cdot\alpha\times height\cdot\beta)$. This can be accounted by the third argument of the `cv2.warpAffine()` function which takes a tuple for the transformed image size.

```
# Image scale
scale = (0.5, 0.5)
T = np.float32([[scale[0], 0, 0],
                [0, scale[1], 0]])
Iscale = cv.warpAffine(I, T, (int(I.shape[1] * scale[0]),
int(I.shape[0] * scale[1])))

# Display it (here we have to show axes to see the image resize
```

```
results)
ShowImages([("Source image", I),
            ("Image scale", Iscale)], 2, hide_axes = False)
```

OpenCV also provides function called `resize()` to resize an image in a single command. We may use it as well.

```
# Image scale
scale = (0.5, 0.5)
Iscale_ocv = cv.resize(I, None, fx = scale[0], fy = scale[1],
interpolation = cv.INTER_CUBIC)

# Display it (here we have to show axes to see the image resize
results)
ShowImages([("Source image", I),
            ("Built-in image scale", Iscale_ocv)], 2, hide_axes =
False)
```

1.3.3.1 Self-work

Self-work

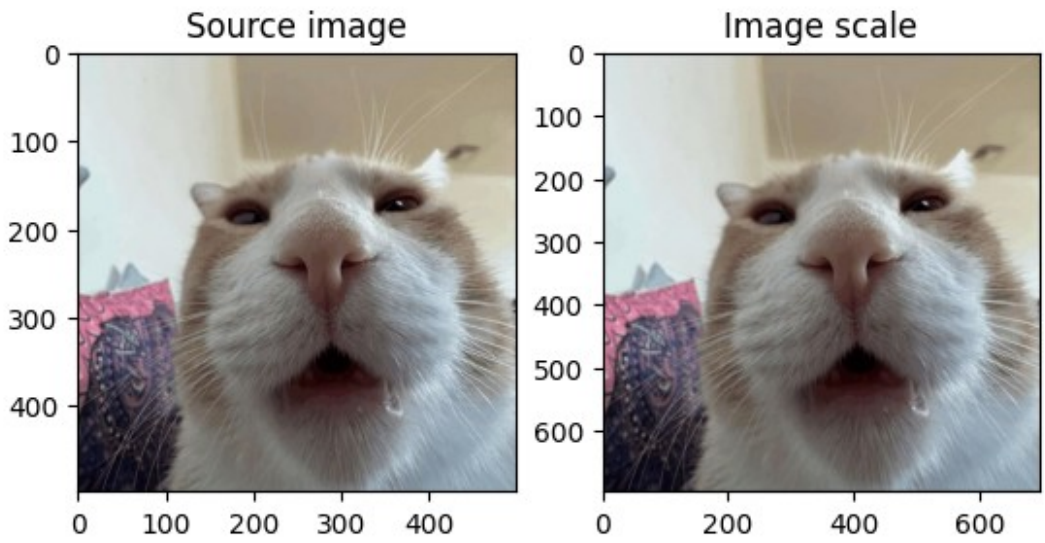
Scale an image to upscale it by 1 and $\frac{2}{5}$ times (scale factor 1.4 along both axis).

Implement two solutions:

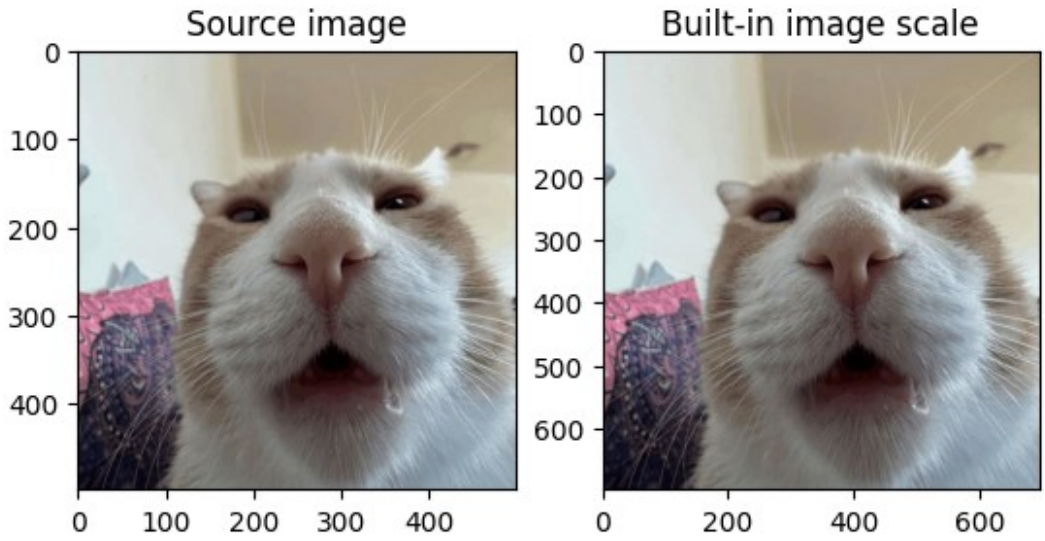
1. Create the transformation matrix and apply it with `cv2.warpAffine()` function;
2. Use the OpenCV built-in `scale()` function.

Compare the results, they should match. Don't forget that this transformation should change the image size.

```
scale2 = (1.4, 1.4)
T_scale = np.float32([[scale2[0], 0, 0],
                      [0, scale2[1], 0]])
Iscale2 = cv.warpAffine(I_cat, T_scale, (int(I_cat.shape[1] *
scale2[0]), int(I_cat.shape[0] * scale2[1])))

ShowImages([("Source image", I_cat),
            ("Image scale", Iscale2)], 2, hide_axes = False)
```




```
Iscale15_ocv = cv.resize(I_cat, None, fx = scale2[0], fy = scale2[1],
interpolation = cv.INTER_CUBIC)

ShowImages([("Source image", I_cat),
            ("Built-in image scale", Iscale15_ocv)], 2, hide_axes =
False)
```



Did your rotation results match? Don't forget to check it. There might be a little difference due to a different floating point values processing.

1.3.4 Image rotation

The transformation equation and coordinates transformation matrix T parameters in case of *image rotation* by φ degree clockwise around the coordinate system center are $A=\cos\varphi$, $B=-\sin\varphi$, $D=\sin\varphi$, $E=\cos\varphi$, $C=F=0$. Then equations and matrix will take the form:

$$\begin{pmatrix} x' = x\cos\varphi - y\sin\varphi \\ y' = x\sin\varphi + y\cos\varphi \end{pmatrix} \Rightarrow T = \begin{bmatrix} \cos\varphi & -\sin\varphi & 0 \\ \sin\varphi & \cos\varphi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

If $\varphi = 90^{\circ}$, then $\cos\varphi = 0$ and $\sin\varphi = 1$ and transformation equation take the form:

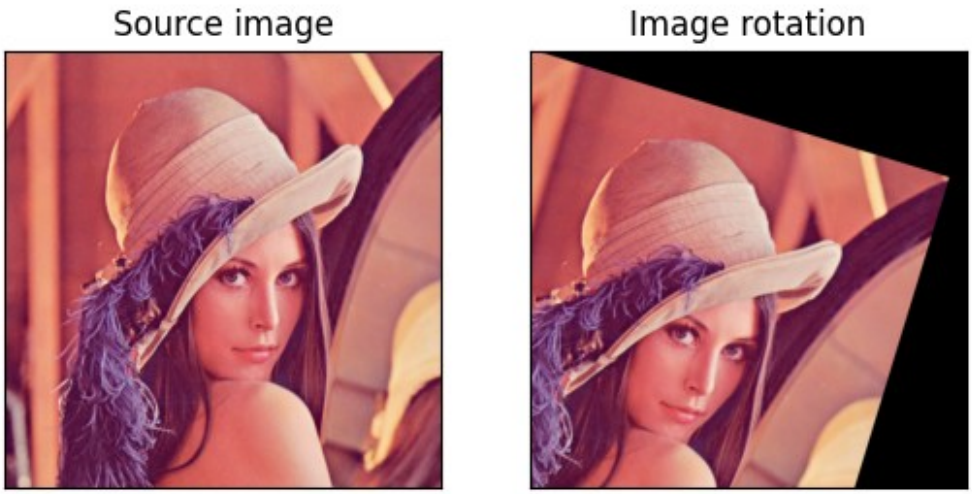
$$\begin{pmatrix} x' = -y \\ y' = x \end{pmatrix} \Rightarrow T = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

In case of using OpenCV libraries the transformation matrix should be defined as:

$$T = \begin{bmatrix} \cos\varphi & -\sin\varphi & 0 \\ \sin\varphi & \cos\varphi & 0 \end{bmatrix}$$

Let's do it.

```
# Image rotation
phi = 17.0 * math.pi / 180 # angle should be defined in radians
T = np.float32([[math.cos(phi),  -math.sin(phi), 0],
                [math.sin(phi),   math.cos(phi), 0]])
Irotate = cv.warpAffine(I, T, I.shape[:2][::-1])

# Display it
ShowImages([("Source image", I), ("Image rotation", Irotate)], 2)
```



The image is really rotated around the coordinate system center which is top left corner, however in general we would like to rotate it around an arbitrary point. To do it, the rotations should be defined as a set of image transformations which are:

1. Shift an image so that the rotation point is in the top left corner;
2. Rotate it with a given angle;
3. Shift it back.

This can be written in a matrix form and then executed as a single affine transformation:

$$T_{shift}=\begin{bmatrix}1 & 0 & -(I.cols-1)/2\\ 0 & 1 & -(I.rows-1)/2\\ 0 & 0 & 1\end{bmatrix}$$

$$T_{rotate}=\begin{bmatrix}\cos\varphi & -\sin\varphi & 0\\ \sin\varphi & \cos\varphi & 0\\ 0 & 0 & 1\end{bmatrix}$$

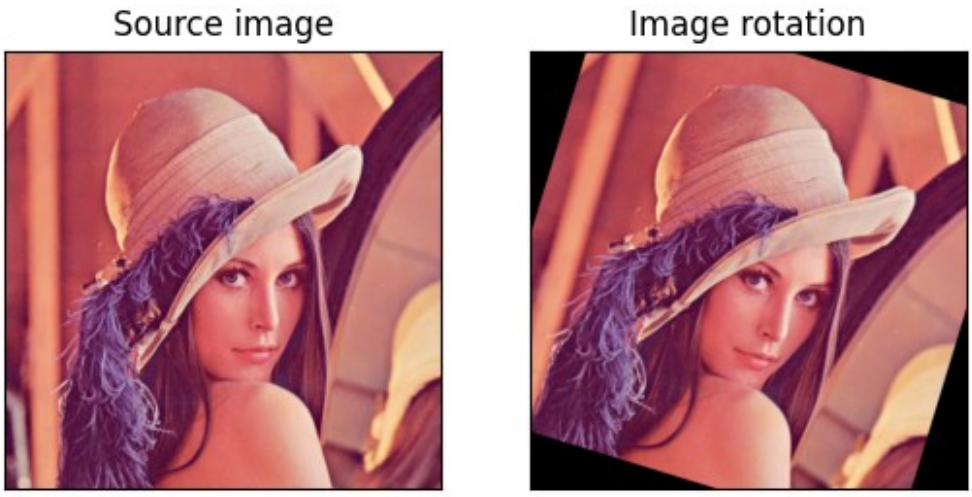
$$T_{shiftback}=\begin{bmatrix}1 & 0 & (I.cols-1)/2\\ 0 & 1 & (I.rows-1)/2\\ 0 & 0 & 1\end{bmatrix}$$

Here T_{shift} matrix is a forward shift to match $(0,0)$ with image center, T_{rotate} is rotation matrix, and $T_{shiftback}$ is backward shift to move image back to place after rotation. Then the final transformation matrix T is calculated by multiplying these three transformation matrices and taking two first rows from it.

$$T=T_{shiftback}\times T_{rotate}\times T_{shift}$$

Let's implement it with OpenCV and rotate our image around its center.

```
# Image rotation around image center
phi = 17.0 * math.pi / 180 # angle should be defined in radians
Tshift = np.float32([[1, 0, -(I.shape[1] - 1) / 2.0],
                     [0, 1, -(I.shape[0] - 1) / 2.0],
                     [0, 0, 1]])
Trotate = np.float32([[math.cos(phi), -math.sin(phi), 0],
                      [math.sin(phi),  math.cos(phi), 0],
                      [0, 0, 1]])
Tshiftback = np.float32([[1, 0, (I.shape[1] - 1) / 2.0],
                         [0, 1, (I.shape[0] - 1) / 2.0],
                         [0, 0, 1]])
T = np.matmul(Tshiftback, np.matmul(Trotate, Tshift))[0:2, :]

Irotate2 = cv.warpAffine(I, T, I.shape[:2][::-1])

# Display it
ShowImages([("Source image", I), ("Image rotation", Irotate2)], 2)
```



Finally, the OpenCV `getRotationMatrix2D()` function can be used to calculate the rotation matrix for counter-clockwise rotation of an image around arbitrary point on an arbitrary angle and scaling.

Let's try it too.

```
# Image rotation around image center
phi = 17.0 # here we use degrees to define an angle
T = cv.getRotationMatrix2D(((I.shape[1] - 1) / 2.0, (I.shape[0] - 1) /
2.0), -phi, 1)
Irotate_ocv = cv.warpAffine(I, T, I.shape[:2][::-1])

# Display it
ShowImages([("Source image", I), ("Built-in image rotation",
Irotate_ocv)], 2)
```

1.3.4.1 Self-work

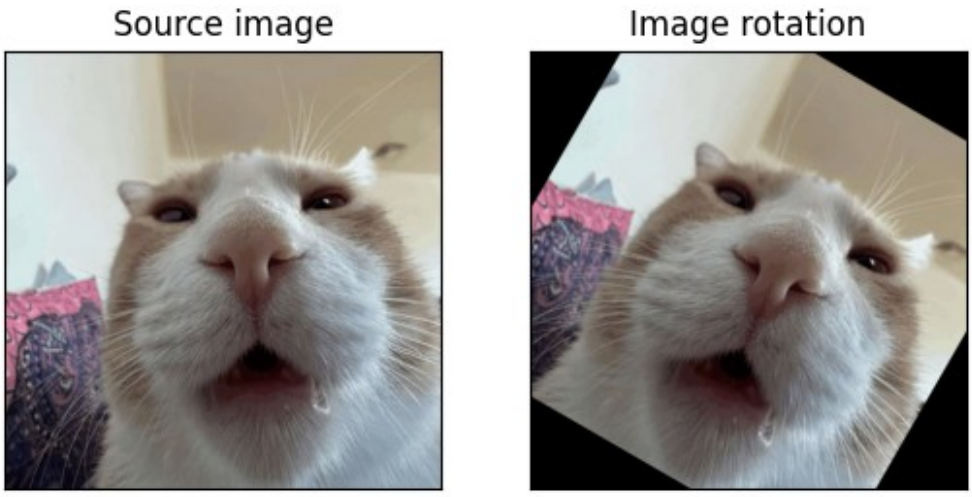
Self-work

Rotate an image around the center of the top right quarter with coordinates $\left(\frac{3}{4}\cdot width, \frac{1}{4}\cdot height\right)$ by $30\degree$ clockwise. Implement two solutions:

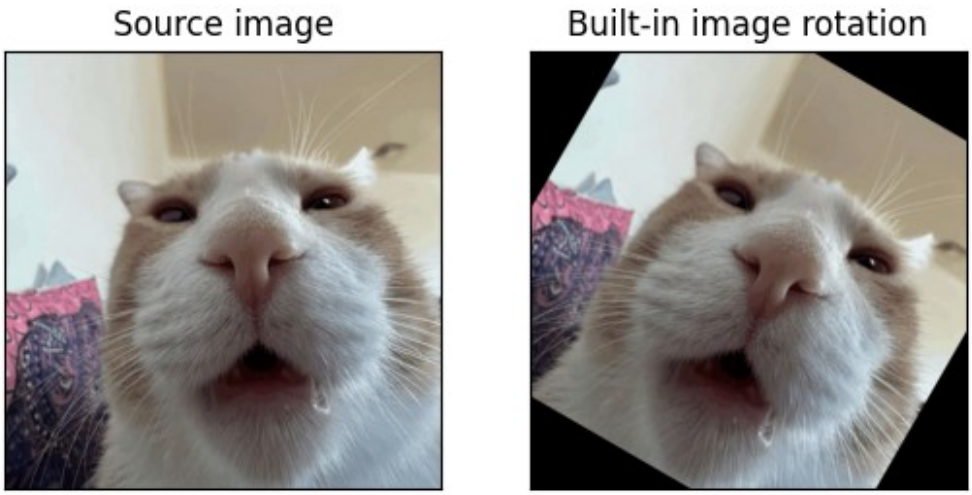
1. Create the rotation matrix manually with three matrices;
2. Use the OpenCV built-in `getRotationMatrix2D()` function.

Compare the results, they should match

```
phi =  math.pi / 6
Tshift2 = np.float32([[1, 0, -(I_cat.shape[1] - 1) / 2.0],
                      [0, 1, -(I_cat.shape[0] - 1) / 2.0],
                      [0, 0, 1]])
Trotate2 = np.float32([[math.cos(phi), -math.sin(phi), 0],
                       [math.sin(phi),  math.cos(phi), 0],
                       [0, 0, 1]])
Tshiftback2 = np.float32([[1, 0, (I_cat.shape[1] - 1) / 2.0],
                          [0, 1, (I_cat.shape[0] - 1) / 2.0],
                          [0, 0, 1]])
T2 = np.matmul(Tshiftback2, np.matmul(Trotate2, Tshift2))[0:2, :]

Irotate2 = cv.warpAffine(I_cat, T2, I_cat.shape[:2][::-1])

ShowImages([("Source image", I_cat), ("Image rotation", Irotate2)], 2)
```




```
phi = 30.0
T3 = cv.getRotationMatrix2D(((I_cat.shape[1] - 1) / 2.0,
(I_cat.shape[0] - 1) / 2.0), -phi, 1)
Irotate_ocv2 = cv.warpAffine(I_cat, T3, I_cat.shape[:2][::-1])

ShowImages([("Source image", I_cat), ("Built-in image rotation",
Irotate_ocv2)], 2)
```



Did your rotation results match? Don't forget to check it. There might be a little difference due to a different floating point values processing.

1.3.5 Arbitrary affine transformation

Affine mapping is a mapping, in which parallel lines go into parallel lines, intersect lines into intersect lines, cross lines into cross lines; the segments lengths ratios of lying on the one straight line (or on parallel lines) are preserved, and the figures areas ratios are preserved also. Basic transformations are linear transformations, bevel and non-uniform scaling. An arbitrary affine transformation can be obtained using the sequential product of the basic transformation matrices. In continuous geometry, any affine transformation has an inverse affine transformation, and the product of the direct and inverse transformations gives a unit transformation that leaves all points in place without changing. Affine transformations are a subset of projection transformations.

Affine mapping

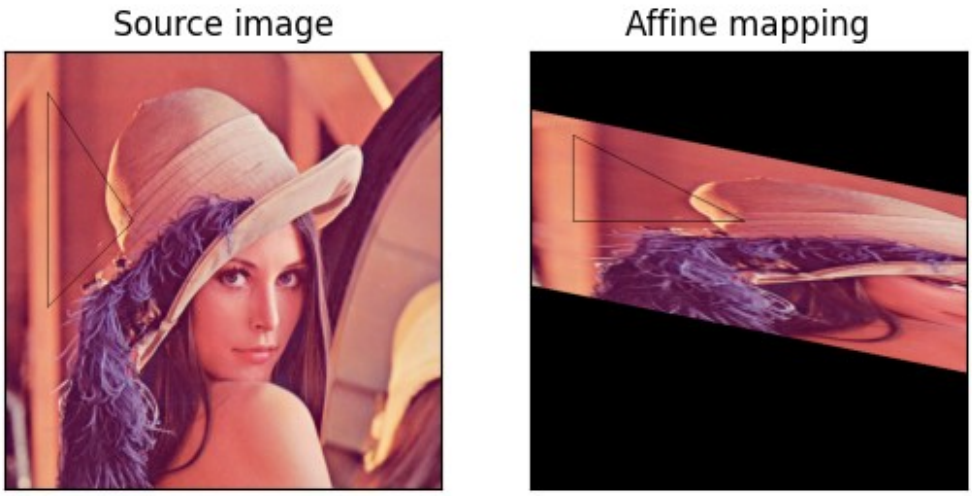
Affine mapping

Such transformation can be defined by three pairs of points and then the transformation matrix will be calculated from a linear equations system. Let us assume we have three pairs of matching points: $P_{src}=\{(x_1,y_1),(x_2,y_2),(x_3,y_3)\}$ which matches $P_{dst}=\{(x'_1,y'_1),(x'_2,y'_2),(x'_3,y'_3)\}$. Then after substitution to the transformation equations we will get:

$$\begin{cases} x'_1=x_1\cdot A+y_1\cdot B+C \\ y'_1=x_1\cdot D+y_1\cdot E+F \\ x'_2=x_2\cdot A+y_2\cdot B+C \\ y'_2=x_2\cdot D+y_2\cdot E+F \\ x'_3=x_3\cdot A+y_3\cdot B+C \\ y'_3=x_3\cdot D+y_3\cdot E+F \end{cases}$$

Which gives us six linear equations with 6 variables. Solving these equations will give us the transformation matrix. To do it, the `getAffineTransform()` function can be used, it takes 3 pairs of points as arguments and returns the corresponding transformation matrix.

```
# Calculate the transformation matrix
Psrc = np.float32([[50, 300], [150, 200], [50, 50]])
Pdst = np.float32([[50, 200], [250, 200], [50, 100]])
T = cv.getAffineTransform(Psrc, Pdst)
Iaffine = cv.warpAffine(I, T, I.shape[:2][::-1])

# Draw triangles on source and transformed images
Icopy = I.copy()
Iaffine_copy = Iaffine.copy()
cv.polylines(Icopy, [Psrc.astype(np.int32)], True, (0, 0, 0), 1)
cv.polylines(Iaffine_copy, [Pdst.astype(np.int32)], True, (0, 0, 0),
1)

# Display it
ShowImages([("Source image", Icopy), ("Affine mapping",
Iaffine_copy)], 2)
```



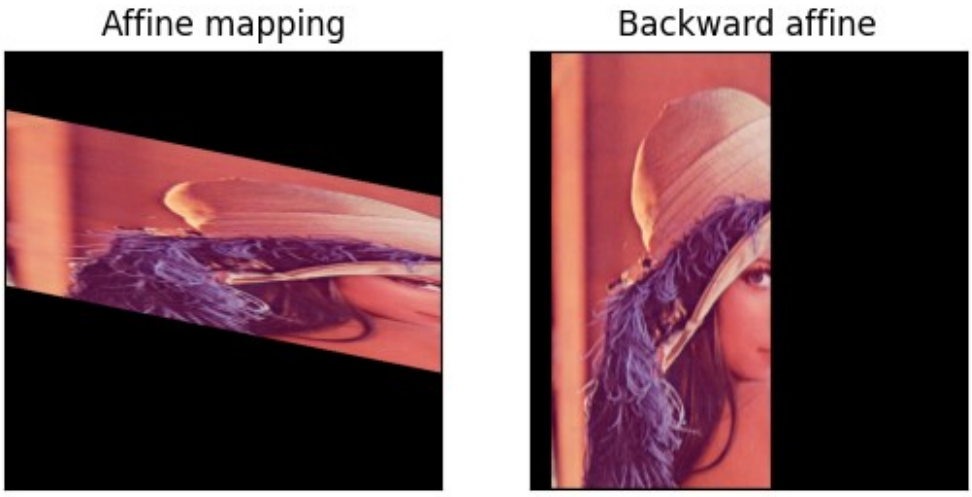
1.3.5.1 Self-work

Self-work

Implement the backward transformation of the `Iaffine` image. The transformation should use `Pdst` points as a source data and `Psrc` points as the destination. Transformed image should match the source image, however some areas may be black due to a data loss.

```
T_back = cv.getAffineTransform(Pdst, Psrc)

Iaffine_back = cv.warpAffine(Iaffine, T_back, I.shape[:2][::-1])
```

```
ShowImages([("Affine mapping", Iaffine), ("Backward affine",
Iaffine_back)], 2)
```



1.3.6 Image bevel

The transformation equation and coordinates transformation matrix T parameters in case of *image bevel* by factor s along $o\,X$ axis are $A=E=1$, $B=s$, $C=D=F=0$. Then equations and matrix will take the form:

$$\left\{\begin{matrix} x'=x+s\,y, \\ y'=y, \end{matrix}\right\}\Rightarrow T=\begin{bmatrix} 1 & 0 & 0 \\ s & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

With OpenCV this will give us the following transformation matrix:

$$T=\begin{bmatrix} 1 & 0 & 0 \\ s & 1 & 0 \end{bmatrix}$$

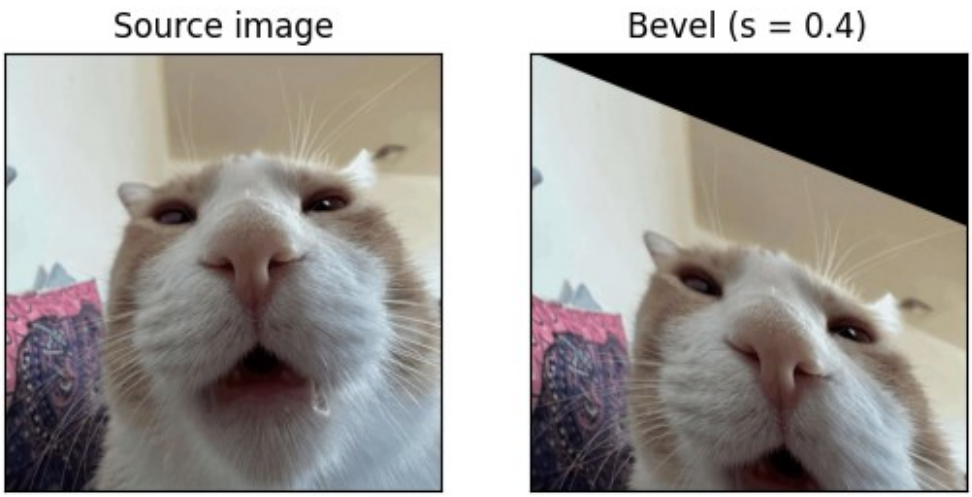
We know how to implement it. Let's do it.

1.3.6.1 Self-work

Self-work

Implement the image bevel transformation according with the matrix above with $s=0.4$ and run it.

```
s = 0.4
T = np.float32([[1, 0, 0],
                [s, 1, 0]])

Ibevel = cv.warpAffine(I_cat, T, I_cat.shape[:2][::-1])

ShowImages([("Source image", I_cat), ("Bevel (s = 0.4)", Ibevel)], 2)
```



1.3.7 Piecewise linear mapping

In some cases we don't need to transform the whole image. *Piecewise-Linear Mapping* is a mapping in which the image is split into parts, and then various linear transformations are applied to each of these parts separately.

When using OpenCV the piecewise operation can be performed with creation of ROI (Region of Interest) image. This ROI type of image shares the data with initial one but can be used as a completely separate image. In Python the ROI for an image is implemented using numpy array indexing and slicing.

Let's try using piecewise linear mapping to scale the right half of an image.

```
# Fill the transformation matrix
stretch = 2
T = np.float32([[stretch, 0, 0],
                [0,       1, 0]])

# Split and display image parts
Ileft = I[:, 0:int(I.shape[1] / 2), :]
Iright = I[:, int(I.shape[1] / 2):, :]
ShowImages([("Left part", Ileft), ("Right part", Iright)], 2)

# Transform right part and stitch
Iright = cv.warpAffine(Iright, T, (int(Iright.shape[1] * stretch),
Iright.shape[0]))
Iplm = np.concatenate((Ileft, Iright), axis = 1)

# Display the result
ShowImages([("Source image", I),
            ("Piecewise linear mapping", Iplm)], 2, hide_axes = False)
```

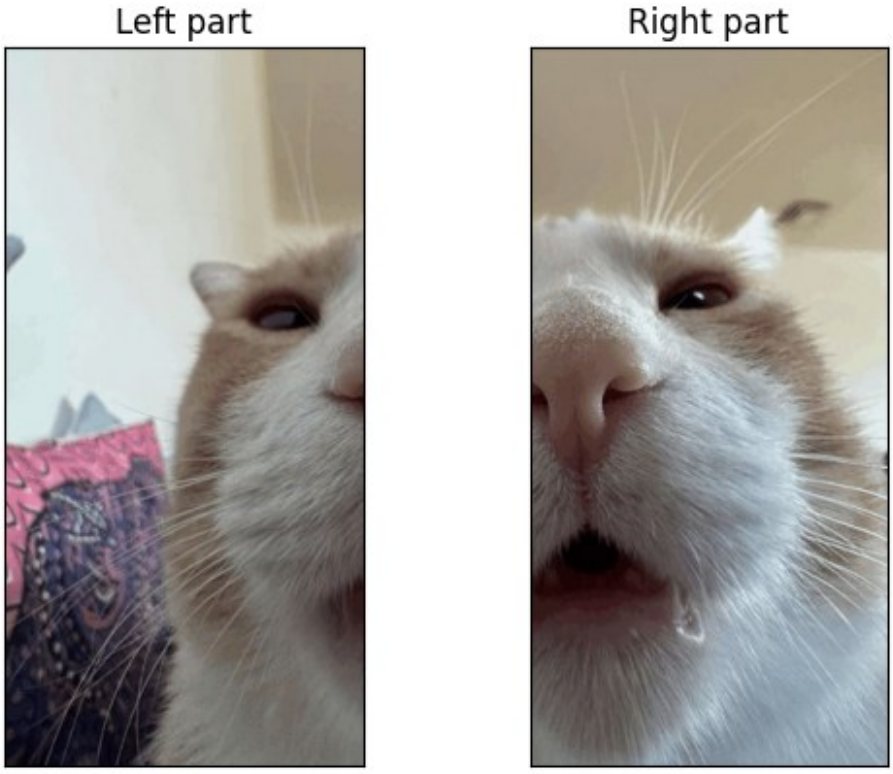
1.3.7.1 Self-work

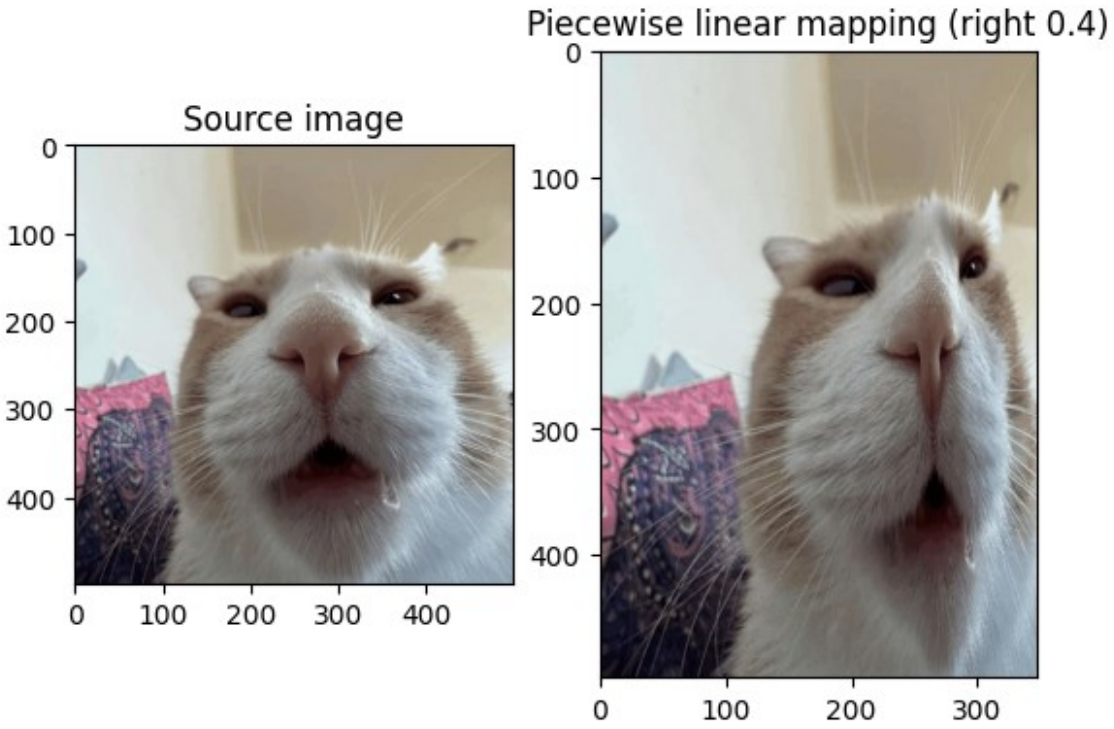
Self-work

Implement the piecewise linear mapping transformation and scale the right part of an image with scale factor 0.4.

```
scale_r = 0.4
T = np.float32([[scale_r, 0, 0],
                [0,       1, 0]])
```

```
Ileft = I_cat[:, :int(I_cat.shape[1] / 2), :]
Iright = I_cat[:, int(I_cat.shape[1] / 2):, :]

ShowImages([("Left part", Ileft), ("Right part", Iright)], 2)

Iright_scaled = cv.warpAffine(Iright, T, (int(Iright.shape[1] *
scale_r), Iright.shape[0]))
Iplm2 = np.concatenate((Ileft, Iright_scaled), axis=1)

ShowImages([("Source image", I_cat), ("Piecewise linear mapping (right
0.4)", Iplm2)], 2, hide_axes=False)
```





1.4 Nonlinear transformations

When considering geometric transformations, it is assumed that the images were obtained using an ideal camera model. In fact, imaging is accompanied by various nonlinear distortions. So, the nonlinear functions are used to correct them.

Nonlinear distortion examples

Nonlinear distortion examples

1.4.1 Perspective mapping

Perspective mapping is a mapping, in which straight lines remain straight lines, however, the figure geometry may be distorted, because this mapping generally does not preserve the lines parallelism. The property preserved under this mapping is points *collinearity*: three points lying on one straight line (collinear) remain on one straight line after mapping. Projection mapping can be as *parallel* (scale is changed), and *perspective* (the figure geometry is changed).

Perspective mapping

Perspective mapping: straight lines remain straight, but parallel ones may intersect

In the case of a perspective mapping a three-dimensional scene point P^3 projected onto a two-dimensional image plane P^2. This mapping $P^3 \rightarrow P^2$ maps the scene Euclidean point $P=(x, y, z)$ (in homogeneous coordinates (x', y', z', w')) in the image point $X=(x, y)$ (in homogeneous coordinates (x', y', w')). To find the points Cartesian coordinates from homogeneous coordinates, we use the following relations: $P=\left(\frac{x'}{w'}, \frac{y'}{w'}, \frac{z'}{w'}\right)$ --- are coordinates of the vector

$\vec{P}$ and $X=\left(\frac{x'}{w'},\frac{y'}{w'}\right)$ --- are coordinates of the vector $\vec{X}$. Substituting in~(\ref{baseTransformHomo}) $w=1$ for vector $\vec{X}$ we obtain the equations system:

$$\left\{\begin{array}{l} x'=\frac{A\,x+B\,y+C}{G\,x+H\,y+I}, \\ y'=\frac{D\,x+E\,y+F}{G\,x+H\,y+I}, \end{array}\right. \Rightarrow T=\begin{bmatrix} A & B & C \\ D & E & F \\ G & H & 1 \end{bmatrix}$$

Due to the coordinates normalization to w' in general, the projection mapping is nonlinear.

With OpenCV we can define projective transformation matrix same as we did for the affine transformtaion, then use the `cv.warpPerspective()` function to apply it.

```
# Perspective transformation
T = np.float32([[1.1, 0.35, 0], [0.2, 1.1, 0], [0.00075, 0.0005, 1]])
Ipersp = cv.warpPerspective(I, T, I.shape[:2][::-1])

# Display it
ShowImages([("Source image", I), ("Perspective transformation",
Ipersp)], 2)
```

OpenCV also provides a handy `cv.getPerspectiveTransform()` function for the projective transformation matrix calculation for an arbitrary projection mapping can be calculated by specifying the coordinates of four source points $P_{src}=\{(x_1,y_1),(x_2,y_2),(x_3,y_3),(x_4,y_4)\}$ and their coordinates after transformation $P_{dst}=\{(x'_1,y'_1),(x'_2,y'_2),(x'_3,y'_3),(x'_4,y'_4)\}$. It works a way similar as calculation of an affine trnasformation matrix: solves the system of linear equations.

Let's try it.

```
# Perspective transformation
Psrc = np.float32([[50, 461], [461, 461], [461, 50], [50, 50]])
Pdst = np.float32([[50, 461], [461, 440], [450, 10], [100, 50]])
T = cv.getPerspectiveTransform(Psrc, Pdst)
Ipersp2 = cv.warpPerspective(I, T, I.shape[:2][::-1])

# Draw points on source and transformed images
Icopy = I.copy()
cv.polylines(Icopy, [Psrc.astype(np.int32)], True, (0, 0, 0), 1)
cv.polylines(Ipersp2, [Pdst.astype(np.int32)], True, (0, 0, 0), 1)

# Display it
ShowImages([("Source image", Icopy), ("Built-in perspective
transformation", Ipersp2)], 2)
```

1.4.1.1 Self-work

Self-work

Implement the backward perspective transformation of the `Ipersp2` image. The transformation should use `Pdst` points as a source data and `Psrc` points as the destination. Transformed image should match the source image, however some areas may be black due to a data loss.

```
T_back = cv.getPerspectiveTransform(Pdst, Psrc)
Ipersp3 = cv.warpPerspective(Ipersp2, T_back, I.shape[:2][::-1])

ShowImages([("Perspective mapping", Ipersp2), ("Backward perspective",
Ipersp3)], 2)
```

1.4.2 Polynomial mapping

Polynomial mapping is an original image mapping using polynomials. In this case, the coordinate transformation matrix T will contain the polynomials coefficients of the corresponding orders for the coordinates x and y. For example, in case of polynomial mapping of the *second order* equations system takes the following form:

$$\begin{cases} x' = a_1 + a_2 x + a_3 y + a_4 x^2 + a_5 x y + a_6 y^2 \\ y' = b_1 + b_2 x + b_3 y + b_4 x^2 + b_5 x y + b_6 y^2 \end{cases}$$

Where x, y are the point coordinates in one coordinate system, and x', y' are points coordinates in another coordinate system, $a_1 \dots a_6$, $b_1 \dots b_6$ are the mapping coefficients.

OpenCV has no built-in function for such kind of transformatios, and indexing array pixel by pixel is very ineffective when wring program with Python programming language. Instead of this two arrays for source and destination indexes should be calculated to copy data from one array to another in a single command. To create re-indexing arrays with Python, a `numpy.meshgrid()` array constructor is used. It creates two arrays to store X and Y coordinates for each image pixel. Then these coordinates are transformed to be used when mapping new image pixels with old ones. The `mask` is used to exclude pixels that became out of range after transformation.

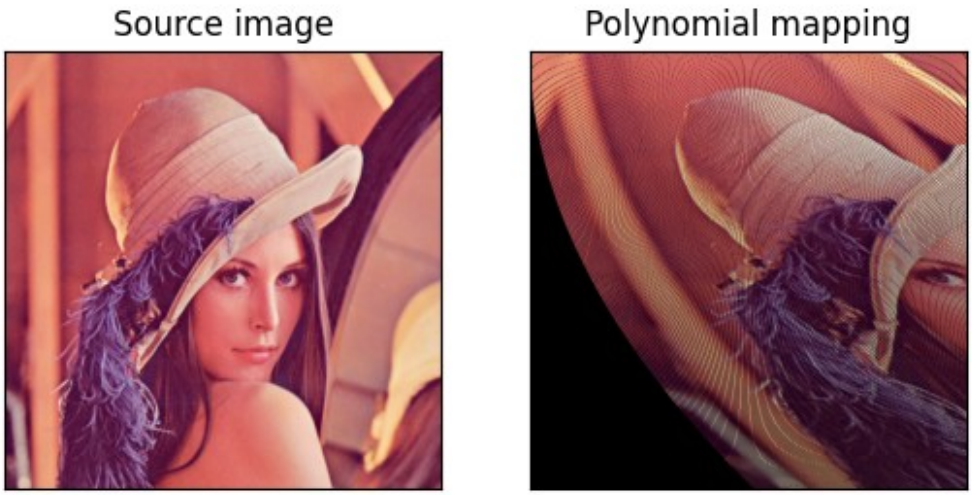
Let's implement a polynomial mapping with a given transformation coefficient as a matrix T using OpenCV:

$$x' = 0 + 1 \cdot x + 0 \cdot y + 0.00001 \cdot x^2 + 0.002 \cdot x y + 0.001 \cdot y^2$$

$$y' = 0 + 0 \cdot x + 1 \cdot y + 0 \cdot x^2 + 0 \cdot x y + 0 \cdot y^2$$

```
# Polynomial mapping
T = np.array([[0, 0], [1, 0], [0, 1], [0.00001, 0],   [0.002, 0],
[0.001, 0]])
# x' = a0 + a1 x + a2 y + a3 x^2 + a4 xy + a5 y^2
# y' = b0 + b1 x + b2 y + b3 x^2 + b4 xy + b5 y^2
# where
```

```
# a = T[:, 0]
# b = T[:, 1]

# Create mesh grid for X, Y coordinates
x, y = np.meshgrid(np.arange(I.shape[1]), np.arange(I.shape[0]))

# Calculate new coordinates for X, Y
x_new = np.round(T[0, 0] + x * T[1, 0] + y * T[2, 0] + x ** 2 * T[3,
0] + x * y * T[4, 0] + y ** 2 * T[5, 0]).astype(np.float32)
y_new = np.round(T[0, 1] + x * T[1, 1] + y * T[2, 1] + x ** 2 * T[3,
1] + x * y * T[4, 1] + y ** 2 * T[5, 1]).astype(np.float32)

# Create a mask where new coordinates are valid
mask = np.logical_and(np.logical_and(x_new >= 0, x_new < I.shape[1]),
                      np.logical_and(y_new >= 0, y_new < I.shape[0]))

# Apply new coordinates
Ipoly = np.zeros(I.shape, I.dtype)
if I.ndim == 2:
  Ipoly[y_new[mask].astype(int), x_new[mask].astype(int)] = I[y[mask],
x[mask]]
else:
  Ipoly[y_new[mask].astype(int), x_new[mask].astype(int), :] =
I[y[mask], x[mask], :]

# Display it
ShowImages([("Source image", I), ("Polynomial mapping", Ipoly)], 2)
```



Since not every pixel on the new image has the corresponding pixel color value the black areas appeared. It can be converted by interpolating the missing pixel values using the known ones.

1.4.2.1 Self-work

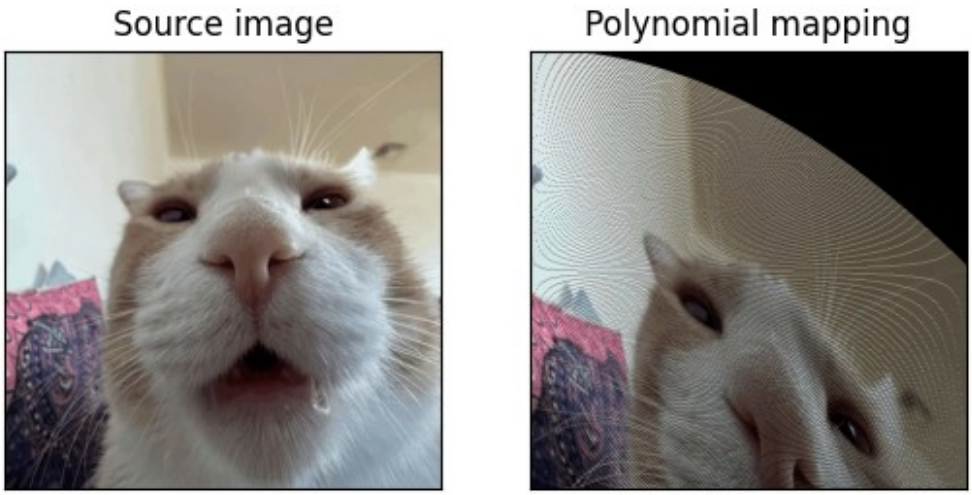
Self-work

Implement the polynomial mapping of the source image with following formula:

$$x' = 0 + 1 \cdot x + 0 \cdot y + 0 \cdot x^2 + 0 \cdot x\,y + 0 \cdot y^2$$

$$y' = 0 + 0 \cdot x + 1 \cdot y + 0.001 \cdot x^2 + 0.002 \cdot x\,y + 0.00001 \cdot y^2$$

```
# x' = a0 + a1 x + a2 y + a3 x^2 + a4 x y + a5 y^2
# y' = b0 + b1 x + b2 y + b3 x^2 + b4 x y + b5 y^2
T = np.array([
    [0.0,      0.0], # a0, b0
    [1.0,      0.0], # a1, b1
    [0.0,      1.0], # a2, b2
    [0.0,      0.001], # a3, b3
    [0.0,      0.002], # a4, b4
    [0.0,      0.00001] # a5, b5
], dtype=np.float32)

x, y = np.meshgrid(np.arange(I_cat.shape[1]),
np.arange(I_cat.shape[0]))

x_new = np.round(
    T[0, 0] + x * T[1, 0] + y * T[2, 0] + x ** 2 * T[3, 0] + x * y *
T[4, 0] + y ** 2 * T[5, 0]
).astype(np.float32)

y_new = np.round(
    T[0, 1] + x * T[1, 1] + y * T[2, 1] + x ** 2 * T[3, 1] + x * y *
T[4, 1] + y ** 2 * T[5, 1]
).astype(np.float32)

mask = np.logical_and(
    np.logical_and(x_new >= 0, x_new < I_cat.shape[1]),
    np.logical_and(y_new >= 0, y_new < I_cat.shape[0])
)

Ipoly2 = np.zeros_like(I_cat)
if I_cat.ndim == 2:
    Ipoly2[y_new[mask].astype(int), x_new[mask].astype(int)] =
I_cat[y[mask], x[mask]]
else:
    Ipoly2[y_new[mask].astype(int), x_new[mask].astype(int), :] =
I_cat[y[mask], x[mask], :]

ShowImages([("Source image", I_cat), ("Polynomial mapping", Ipoly2)],
2)
```

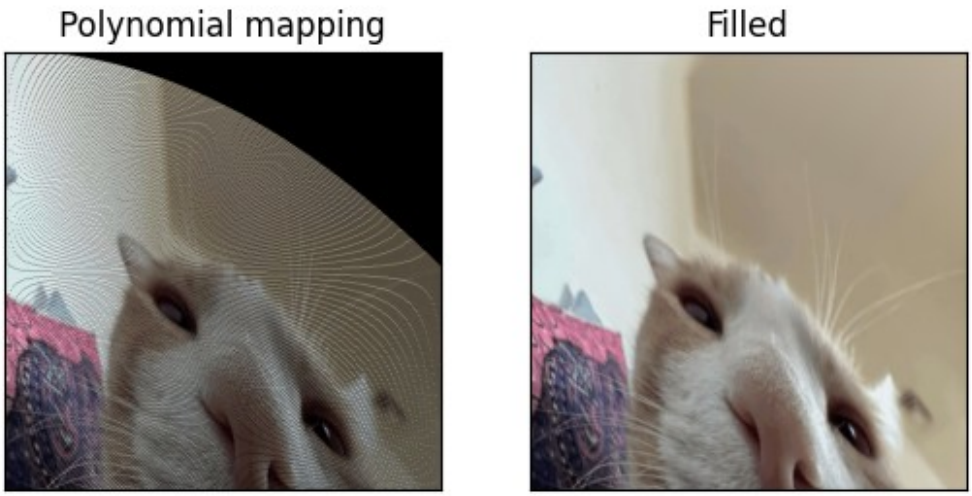


1.4.2.2 Self-work (Optional)

***Self-work** (Optional)*

After applying the polynomial mapping, pixels with undefined intensity values appeared. Update algorithm implementation to fill the pixels with undefined intensity.

```
if Ipoly2.ndim == 3:
    holes = np.all(Ipoly2 == 0, axis=2).astype(np.uint8) * 255
else:
    holes = (Ipoly2 == 0).astype(np.uint8) * 255

# Inpaint expects 8-bit 1-channel mask
Ipoly2_filled = cv.inpaint(Ipoly2, holes, 3, cv.INPAINT_TELEA)

ShowImages([("Polynomial mapping", Ipoly2), ("Filled",
Ipoly2_filled)], 2)
```



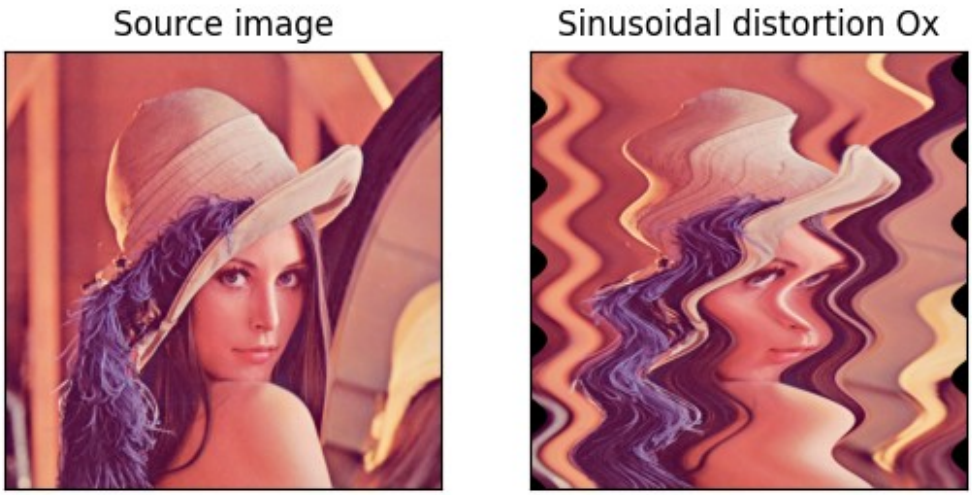
1.4.3 Sinusoidal distortion

Harmonic distortion of an image can be considered as another example of nonlinear transformation.

Let's try creating the sinusoidal distortion. For this we will use OpenCV remapping feature whith allows to remap the whole image by defining the pixel coordinates in the source coordinate system for each pixel of the transformed image. This is implemented by the `remap()` function which transforms image according to new pixel coordinates specified for each pixel of the source image.

Let's apply the sinusoidal distortion along $O\,x$ axis.

```
# Create mesh grid for X, Y coordinates
x, y = np.meshgrid(np.arange(I.shape[1]), np.arange(I.shape[0]))

# Distort the X coordinate depending on Y
x = x + 20 * np.sin(2 * math.pi * y / 90)

# Remap with new coordinates
Isin_x = cv.remap(I, x.astype(np.float32), y.astype(np.float32),
cv.INTER_LINEAR)

# Display it
ShowImages([("Source image", I), ("Sinusoidal distortion Ox",
Isin_x)], 2)
```



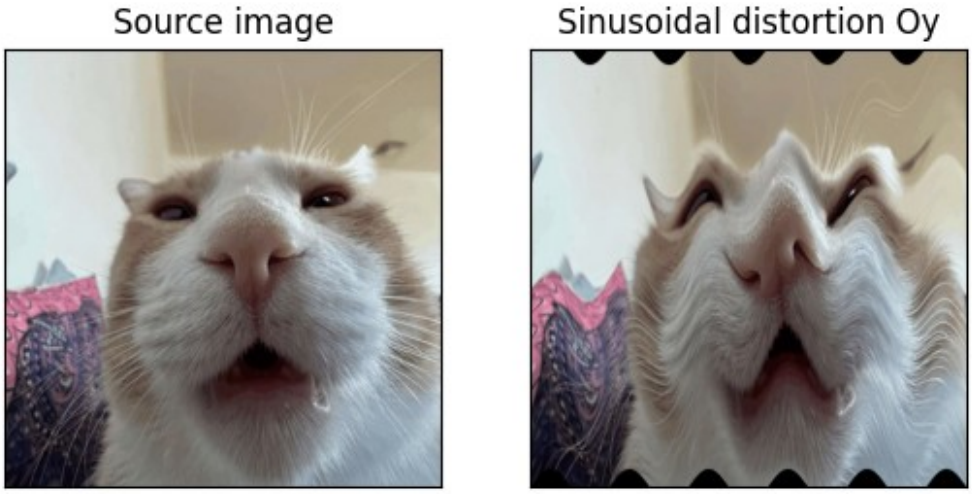
1.4.3.1 Self-work

Self-work

Implement the sinusoidal distortion along $O\,y$ axis with same parameter values.

```
x, y = np.meshgrid(np.arange(I_cat.shape[1]),
np.arange(I_cat.shape[0]))
```

```
y = y + 20 * np.sin(2 * math.pi * x / 90)

Isin_y = cv.remap(I_cat, x.astype(np.float32), y.astype(np.float32),
cv.INTER_LINEAR)

ShowImages([("Source image", I_cat), ("Sinusoidal distortion Oy",
Isin_y)], 2)
```



Task 2. Distortion correction

Select an arbitrary image with either pincushion or barrel distortion. Correct the image.

When the optical system is forming an image, *distortion* may occur on the image.

Distortion is an optical distortion that is expressed in the curvature of straight lines. Light rays passing through the center of the lens converge at a point farther from the lens than rays passing through the edge of the lens. Straight lines are curved, except for those that lie in the same plane with the optical axis. For example, the image of a square, the center of which intersects the optical axis, looks like a pillow (pincushion distortion*) with positive distortion and looks like a barrel (*barrel distortion*) with negaitive distortion.

Distortions examples

Distortions examples. Left is the original image, center is a pincushion distortion, right is a barrel distortion

Let $\vec{r} = (x, y)$ --- vector specifying two coordinates in a plane perpendicular to the optical axis. For an ideal image, all rays leaving this point and passing through the optical system will hit the image point with coordinates $\vec{R}$, which are determined by the formula:

$$\vec{R} = b_0 \vec{r}$$

Where b_0 is a linear increasing parameter.

If distortion of a higher order is present (for axisymmetric optical systems, the distortion can only be of odd orders: third, fifth, seventh, etc.), then it is necessary to add the appropriate terms:

$$\vec{R}=b_0\vec{r}+F_3r^2\vec{r}+F_5r^4\vec{r}+\dots$$

where r is the vector length $\vec{r}$; $F_i, i=3,5,\dots,n$ are the distortion coefficients of n-th order, which contribute the most to the distortion of the image shape. With third-order distortion, if the coefficient F_3 has the same sign as b_0: $sign(F_3)=sign(b_0)$, pincushion distortion occurs, otherwise a barrel distortion occurs.

To correct the distortion, the approach described above for affine of projective transformation is used. An image of a regular grid and its distorted image is used, pairs of points on these images are found, and the coefficients of the corrective transformation are calculated.

2.1 Barrel distortion

To distort an image we will do the following steps:

1. First, create a `meshgrid` for our image pixel coordinates;
2. Then shift the mesh coordinates so that image center has coordinates $(0,0)$ and transform them to a polar coordinate system;
3. Apply distortion transformation in the polar coordinate system;
4. Convert mesh coordinates back to cartesian coordinate system and shift back;
5. Remap an image with the calculated mesh map.

Let's distort an image according to formula:

$$\vec{R}=\vec{r}+0.1\cdot r^2\vec{r}+0.12\cdot r^4\vec{r}$$

```
# Create mesh grid for X, Y coordinates
x, y = np.meshgrid(np.arange(I.shape[1]), np.arange(I.shape[0]))

mid = I.shape[1] / 2.0, I.shape[0] / 2.0
x, y = x - mid[0], y - mid[1]

r, theta = cv.cartToPolar(x / mid[0], y / mid[1])
b0 = 1
F3 = 0.2
F5 = 0.12
r = b0 * r + F3 * r ** 3 + F5 * r ** 5
#theta += 10 * 3.14 / 180
u, v = cv.polarToCart(r, theta)
u = u * mid[0] + mid[0]
v = v * mid[1] + mid[1]

Ibarrel = cv.remap(I, u.astype(np.float32), v.astype(np.float32),
cv.INTER_LINEAR)
```

```
# Display it
ShowImages([("Source image", I), ("Barrel distortion", Ibarrel)], 2)
```

To correct the distortion we need to define a set of matching points on both distored and corrected image. We will calculate these points along radius in polar coordinate system assuming they were defined by user.

```
# Calculate the set of matching points
d = 5  # Number of points
max_r = 1.3  # points array scale
Psrc = (np.arange(d, dtype = np.float32) + 1) / d * max_r
Pdst = b0 * Psrc + F3 * Psrc ** 3 + F5 * Psrc ** 5

# Now create and fill matrix for linear equations
T = np.zeros((d, d))
for i in range(d):
  T[:, i] = Pdst ** (i + 1)
# Invert it and calculate coefficients
T = np.linalg.inv(T)
F = np.matmul(T, Psrc)

# Now use the calculated parameters to calculate the new mapping
# x and y are already shifted, so we don't need to repeat it
r, theta = cv.cartToPolar(x / mid[0], y / mid[1])
rt = 0
for i in range(d):
  rt = rt + F[i] * r ** (i + 1)
r = rt

# Now convert back to cartesian
u, v = cv.polarToCart(r, theta)
u = u * mid[0] + mid[0];
v = v * mid[1] + mid[1];
```

```
# Apply and display
Ibarrel_corr = cv.remap(Ibarrel, u.astype(np.float32),
v.astype(np.float32), cv.INTER_LINEAR)
ShowImages([("Barrel distortion", Ibarrel), ("Distortion correction",
Ibarrel_corr)], 2)
```

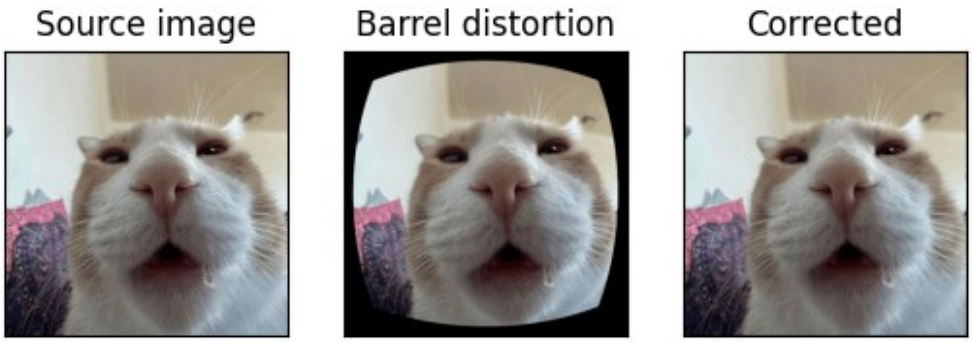
2.1.1 Self-work

Self-work

Distort an image according with formula $\vec{R} = \vec{r} - 0.2 \cdot r^2 \vec{r} + 0.32 \cdot r^4 \vec{r}$ and correct the distortion.

```
H, W = I_cat.shape[:2]

x, y = np.meshgrid(np.arange(W), np.arange(H))
cx, cy = W / 2.0, H / 2.0

x0 = x - cx
y0 = y - cy
xn = x0 / cx
yn = y0 / cy

r, theta = cv.cartToPolar(xn.astype(np.float32),
yn.astype(np.float32))

b0 = 1.0
F3 = -0.2
F5 = 0.32

R = b0 * r + F3 * (r ** 3) + F5 * (r ** 5)

u, v = cv.polarToCart(R, theta)
mapx_dist = (u * cx + cx).astype(np.float32)
```

```
mapy_dist = (v * cy + cy).astype(np.float32)

Ibarrel_sw = cv.remap(I_cat, mapx_dist, mapy_dist, cv.INTER_LINEAR)

x, y = np.meshgrid(np.arange(W), np.arange(H))
x0 = x - cx
y0 = y - cy
xn = x0 / cx
yn = y0 / cy

R_des, theta = cv.cartToPolar(xn.astype(np.float32),
yn.astype(np.float32))

r = R_des.copy().astype(np.float32)
for _ in range(7):
    f  = r + F3 * (r ** 3) + F5 * (r ** 5) - R_des
    df = 1.0 + 3.0 * F3 * (r ** 2) + 5.0 * F5 * (r ** 4)
    r = r - f / (df + 1e-8)

u, v = cv.polarToCart(r, theta)
mapx_und = (u * cx + cx).astype(np.float32)
mapy_und = (v * cy + cy).astype(np.float32)

Icorr_sw = cv.remap(Ibarrel_sw, mapx_und, mapy_und, cv.INTER_LINEAR)

ShowImages([("Source image", I_cat), ("Barrel distortion",
Ibarrel_sw), ("Corrected", Icorr_sw)], 3)
```



2.2 Pincushion distortion

The pincushion distortion is calculated a similar way but with different parameters.

Let's distort an image according to formula:

$$\vec{R} = 1.2 \cdot \vec{r} - 0.15 \cdot r^2 \vec{r}$$

```
# Create mesh grid for X, Y coordinates
x, y = np.meshgrid(np.arange(I.shape[1]), np.arange(I.shape[0]))
```

```
mid = I.shape[1] / 2.0, I.shape[0] / 2.0
x, y = x - mid[0], y - mid[1]

r, theta = cv.cartToPolar(x / mid[0], y / mid[1])
b0 = 1.2
F3 = -0.15
F5 = 0
r = b0 * r + F3 * r ** 3 + F5 * r ** 5
u, v = cv.polarToCart(r, theta)
u = u * mid[0] + mid[0]
v = v * mid[1] + mid[1]

Ipincushion = cv.remap(I, u.astype(np.float32), v.astype(np.float32),
cv.INTER_LINEAR)

# Display it
ShowImages([("Source image", I), ("Pincushion distortion",
Ipincushion)], 2)
```

Let's correct this one as well.

```
# Calculate the set of matching points
d = 5  # Number of points
max_r = 1.4  # points array scale
Psrc = (np.arange(d, dtype = np.float32) + 1) / d * max_r
Pdst = b0 * Psrc + F3 * Psrc ** 3 + F5 * Psrc ** 5

# Now create and fill matrix for linear equations
T = np.zeros((d, d))
for i in range(d):
  T[:, i] = Pdst ** (i + 1)
# Invert it and calculate coefficients
T = np.linalg.inv(T)
```

```
F = np.matmul(T, Psrc)

# Now use the calculated parameters to calculate the new mapping
# x and y are already shifted, so we don't need to repeat it
r, theta = cv.cartToPolar(x / mid[0], y / mid[1])
rt = 0
for i in range(d):
  rt = rt + F[i] * r ** (i + 1)
r = rt

# Now convert back to cartesian
u, v = cv.polarToCart(r, theta)
u = u * mid[0] + mid[0];
v = v * mid[1] + mid[1];

# Apply and display
Ipincushion_corr = cv.remap(Ipincushion, u.astype(np.float32),
v.astype(np.float32), cv.INTER_LINEAR)
ShowImages([("Pincushion distortion", Ipincushion), ("Distortion
correction", Ipincushion_corr)], 2)
```

It can be seen this image is also corrected. Black areas appeared because of data loss when the distortion was applied.

2.2.1 Self-work

Self-work

Distort an image according with formula $\vec{R} = 1.3 \cdot \vec{r} - 0.2 \cdot r^2 \vec{r} + 0.05 \cdot r^4 \vec{r}$ and correct the distortion.

```
x, y = np.meshgrid(np.arange(I_cat.shape[1]),
np.arange(I_cat.shape[0]))

mid = I_cat.shape[1] / 2.0, I_cat.shape[0] / 2.0
```

```
x, y = x - mid[0], y - mid[1]

r, theta = cv.cartToPolar((x / mid[0]).astype(np.float32), (y /
mid[1]).astype(np.float32))
b0 = 1.3
F3 = -0.2
F5 = 0.05
r_d = b0 * r + F3 * r ** 3 + F5 * r ** 5

u, v = cv.polarToCart(r_d, theta)
u = u * mid[0] + mid[0]
v = v * mid[1] + mid[1]

Ipincushion_sw = cv.remap(I_cat, u.astype(np.float32),
v.astype(np.float32), cv.INTER_LINEAR)

R_des, theta = cv.cartToPolar((x / mid[0]).astype(np.float32), (y /
mid[1]).astype(np.float32))

r = (R_des / max(b0, 1e-6)).astype(np.float32)
for _ in range(8):
    f  = b0 * r + F3 * r**3 + F5 * r**5 - R_des
    df = b0 + 3.0 * F3 * r**2 + 5.0 * F5 * r**4
    r = r - f / (df + 1e-8)

u, v = cv.polarToCart(r, theta)
u = u * mid[0] + mid[0]
v = v * mid[1] + mid[1]

Ipincushion_sw_corr = cv.remap(Ipincushion_sw, u.astype(np.float32),
v.astype(np.float32), cv.INTER_LINEAR)

ShowImages([("Source image", I_cat),
            ("Pincushion distortion", Ipincushion_sw),
            ("Distortion correction", Ipincushion_sw_corr)], 3)
```

Task 3. Stitching images

Select two images (photographs from a cam- era, fragments of a scanned image, etc.) which have an overlapping area. Correct the second image to translate it into the coordinate system of the first one; then perform automatic "glueing" from two images into the one.

Geometric transformations can be used, for example, to create mosaics from multiple images. Mosaic "*stitching*", "*gluing*") is the combination of two or more images into a single one, and the images coordinate systems being glued may differ due to different shooting angles, changes in the camera position or the object movement. However, it is necessary that both images have overlapping areas, i.e. they were attended by the same objects.

3.1 Manual image stitching

The main task of processing such images is to bring them into a common coordinate system. As a common coordinate system, you can use the system of the first image, then you need to find the coordinate transformation of all pixels of the second image (x, y) to the common coordinate system (x', y'). If there is a projective distortion, then to recalculate the coordinates, you can use the projective transformation. In the case of an affine mapping, the transformation takes the form:

$$\begin{pmatrix} x' = a_1 + a_2 x + a_3 y, \\ y' = b_1 + b_2 x + b_3 \end{pmatrix}$$

Therefore, it is only necessary to find only 3 parameters for each coordinate:

$$\begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix} = \begin{bmatrix} 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \\ 1 & x_3 & y_3 \end{bmatrix}^{-1} \begin{bmatrix} x'_1 \\ x'_2 \\ x'_3 \end{bmatrix}$$

$$\begin{bmatrix} b_1 \\ b_2 \\ b_3 \end{bmatrix} = \begin{bmatrix} 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \\ 1 & x_3 & y_3 \end{bmatrix}^{-1} \begin{bmatrix} y'_1 \\ y'_2 \\ y'_3 \end{bmatrix}$$

To do this we will search for a correlation between two images. We will take severla bottom rows of the first image and serach for a correlation of these rows in a second image.

In OpenCV the correlation between two images and it is calculated with `matchTemplate()` function. This functions checks correlation with a given template over the whole image and returns an image with size $I.size - template.size$ where foe the `TM_CCOEFF` mode the higher the value is the higher is the correlation rate when matching a template from this point. We need to select the highest correlation point and shift the second image by this value before merging using the ROI image indexing.

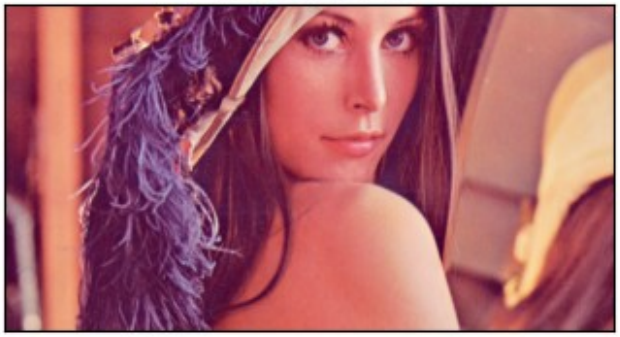
Let's do it.

```
fn_top = "images/lena_top.png";
fn_bottom = "images/lena_bottom.png";
```

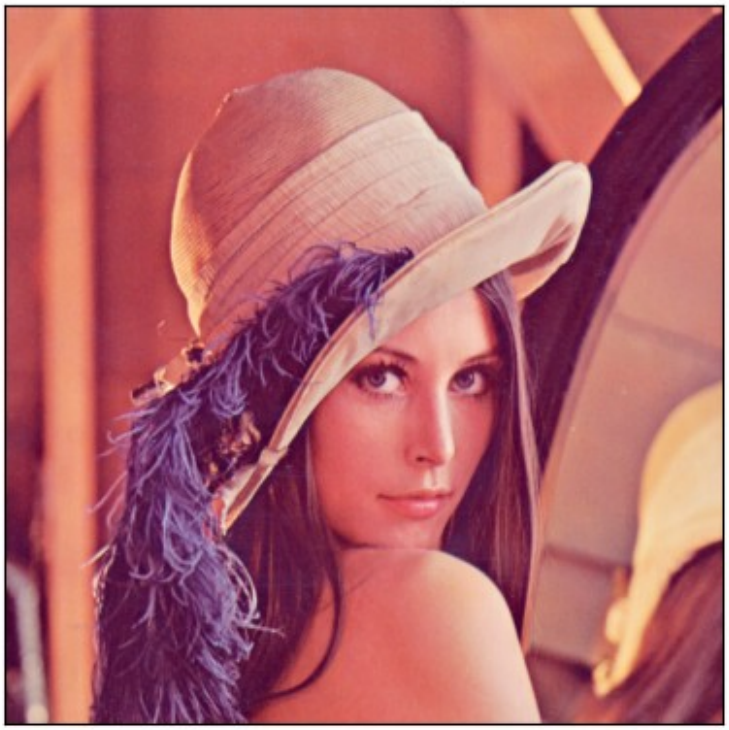
```
# Read an images from files
Itop = cv.imread(fn_top, cv.IMREAD_COLOR)
if not isinstance(Itop, np.ndarray) or Itop.data == None:
  print("Error reading file \"{}\"".format(fn_top))
Ibottom = cv.imread(fn_bottom, cv.IMREAD_COLOR)
if not isinstance(Ibottom, np.ndarray) or Ibottom.data == None:
  print("Error reading file \"{}\"".format(fn_bottom))

# Show parts of image
ShowImages([("Top part", Itop), ("Bottom part", Ibottom)], 1)

# Match template
templ_size = 10
templ = Itop[-templ_size:, :, :]
res = cv.matchTemplate(Ibottom, templ, cv.TM_CCOEFF)
min_val, max_val, min_loc, max_loc = cv.minMaxLoc(res)

# Stitch images
Istitch = np.zeros((Itop.shape[0] + Ibottom.shape[0] - max_loc[1] -
templ_size,
                    Itop.shape[1], Itop.shape[2]), dtype = np.uint8)
Istitch[0:Itop.shape[0], :, :] = Itop
Istitch[Itop.shape[0]:, :, :] = Ibottom[max_loc[1] +
templ_size:, :, :]

# Show stitched image
ShowImages([("Stitched image", Istitch)])
```

Top part

Bottom part



Stitched image



3.1.1 Self-work

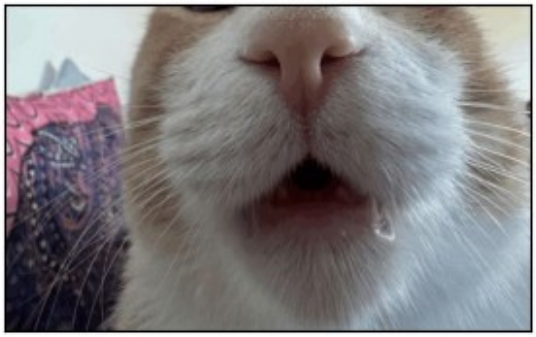
Self-work

Take some image, split it into two parts and try stitching it. You can split an image with OpenCV as well and then stitch it back:

```
Itop = I[0:int(I.shape[0] / 2) + 20, :, :]
Ibottom = I[int(I.shape[0] / 2):, :, :]

H, W = I_cat.shape[:2]
overlap = 60

Itop2 = I_cat[:H//2 + overlap, :, :].copy()
Ibottom2 = I_cat[H//2 - overlap:, :, :].copy()

ShowImages([("Top part", Itop2), ("Bottom part", Ibottom2)], 1)

templ_size = 20
templ = Itop2[-templ_size:, :, :]
res = cv.matchTemplate(Ibottom2, templ, cv.TM_CCOEFF)
_, _, _, max_loc = cv.minMaxLoc(res)

y_shift = max_loc[1] + templ_size

Istitch2 = np.zeros((Itop2.shape[0] + Ibottom2.shape[0] - y_shift,
                     W, I_cat.shape[2]), dtype=np.uint8)
Istitch2[:Itop2.shape[0], :, :] = Itop2
Istitch2[Itop2.shape[0]:, :, :] = Ibottom2[y_shift:, :, :]

ShowImages([("Stitched image", Istitch2)], 1)
```

Top part

Bottom part



Stitched image

3.2 Automatic image stitching

OpenCV library provides a special class for automatic stitching of images called `Stitcher`. It is designed to work either for stitching panoramic photos `cv2.Stitcher_PANORAMA` mode or for stitching scanned images `cv2.Stitcher_SCANS` mode. Working with `Stitcher` object is very simple. First, need to create an object. Then need to create an array of OpenCV images and pass them to the `stitch()` method. It returns a tuple with its status and a stitched image. The status value shows wether the stitching succeeded. In case of success it is equal to `cv2.Stitcher_OK`.

Please note that this method requires at least tree images to work correctly.

Let's try it.

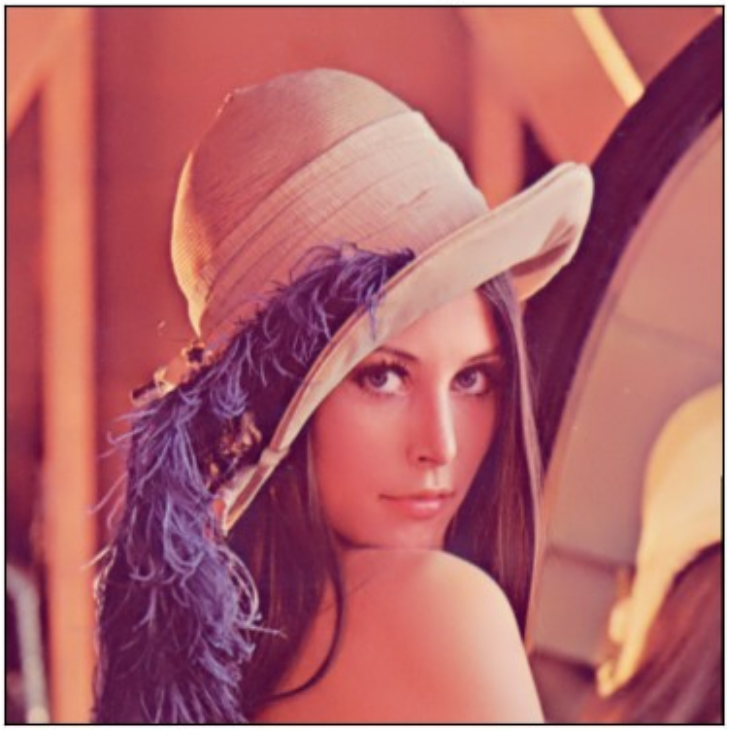
```
fn_top = "images/lena_top.png"
fn_mid = "images/lena_mid.png"
fn_bottom = "images/lena_bottom.png"

# Read an images from files
Itop = cv.imread(fn_top, cv.IMREAD_COLOR)
if not isinstance(Itop, np.ndarray) or Itop.data == None:
  print("Error reading file \"{}\"".format(fn_top))
Imid = cv.imread(fn_mid, cv.IMREAD_COLOR)
if not isinstance(Imid, np.ndarray) or Imid.data == None:
  print("Error reading file \"{}\"".format(fn_mid))
Ibottom = cv.imread(fn_bottom, cv.IMREAD_COLOR)
if not isinstance(Ibottom, np.ndarray) or Ibottom.data == None:
  print("Error reading file \"{}\"".format(fn_bottom))

# Show parts of image
ShowImages([("Top part", Itop), ("Mid part", Imid), ("Bottom part",
Ibottom)], 1)

# Stitch
stitcher = cv.Stitcher.create(cv.Stitcher_SCANS)
status, Istitch2 = stitcher.stitch([Itop, Imid, Ibottom])

# Show stitched image in case of success
if status == cv.Stitcher_OK:
  ShowImages([("Stitched image", Istitch2)])
else:
  print("Stitching error occurred")
```

Top part

Mid part

Bottom part

Stitched image



This OpenCV stitcher object uses more complex stitching algorithm that is based on feature points matching between images, so may not work well for too simple cases.

Questions

Please answer the following questions:

- How can you rotate an image without using a rotation matrix?

  Rotate using **polar coordinates**: convert pixel coordinates (x, y) to (r, θ), add the rotation angle α to the angle $\theta \leftarrow \theta + \alpha$, convert back to (x', y'), and use `cv.remap` (with interpolation) to sample intensities.

- What is the minimum number of corresponding pairs of points that must be specified on the original and distorted images to correct a perspective distortion?

  4 pairs of points (a planar projective transform / homography has 8 degrees of freedom up to scale, so 4 correspondences are sufficient).

- After geometric transformation of the image, pixels with undefined intensity values may appear. What is the reason for this and how can this problem be solved?

  Undefined pixels appear because **forward mapping produces holes**: due to discretization/rounding some destination pixels receive no source value, while others may get multiple mappings.
  Solve by using **inverse mapping** (compute source coordinates for each destination pixel) and **interpolation** (nearest/bilinear/bicubic).

Conclusion

What have you learned with this task? Don't forget to conclude it.

> In this task we learned how to apply and compare the main types of geometric image transformations: affine and perspective (homography) warps, and how to build custom coordinate mappings with `cv.remap`. We practiced working with point correspondences, understood why perspective correction requires at least 4 pairs of points, and saw the difference between forward and inverse mapping and why inverse mapping with interpolation is preferred to avoid “holes”. We also implemented radial lens distortion (barrel/pincushion) and its correction by inverting the distortion model (numerical inversion/Newton), which clarified how real camera distortions can be simulated and compensated in practice.